Ferramentas de Eletrônica e Computação utilizadas em Proteção e Controle - IoT

Angelo Marques Maia, 30/05/2025

Objetivos

- **Entender** conceitos relacionados a inteligência artificial **exemplificando** características e particularidades através de uma implementação em plataforma computacional e **interpretando** o impacto do ajuste de parâmetros nos resultados obtidos.
- **Analisar** resultados obtidos através do treinamento de modelos **comparando** diferentes casos.
- **Avaliar** o desempenho dos modelos **checando e criticando** os ajustes realizados
- **Aplicar** o conteúdo apresentado **implementando** um modelo de aprendizado de máquina em um microcontrolador

Índice

1. Apresentação do Tema
2. Apresentação da Plataforma
3. Atividade
    - 3.1. Criação do projeto
    - 3.2. Ajustes dos arquivos de entrada
    - 3.3. Alimentação da plataforma com dados de entrada
    - 3.4. Ajustes de parâmetros
    - 3.5. Verificação dos resultados
    - 3.6. Teste dentro da plataforma
    - 3.7. Como embarcar o modelo em microcontrolador
    - 3.8. Teste em microcontrolador

Aprendizado de Máquina

- Objetivo: **identificar padrões e tomar decisões** a partir de **sinais de entrada**;
- Algumas aplicações:
  - Identificação de falhas;
  - Automação de processos;
  - Relacionamento com clientes;
  - Detecção de fraudes bancárias;

Aprendizado de Máquina

- **Padrões**
  - Variação abrupta no sinal (defeitos);
  - Frequências e amplitudes em áudio (vozes, palavras específicas);
  - Padrões em imagens (objetos, detalhes).

Aprendizado de Máquina

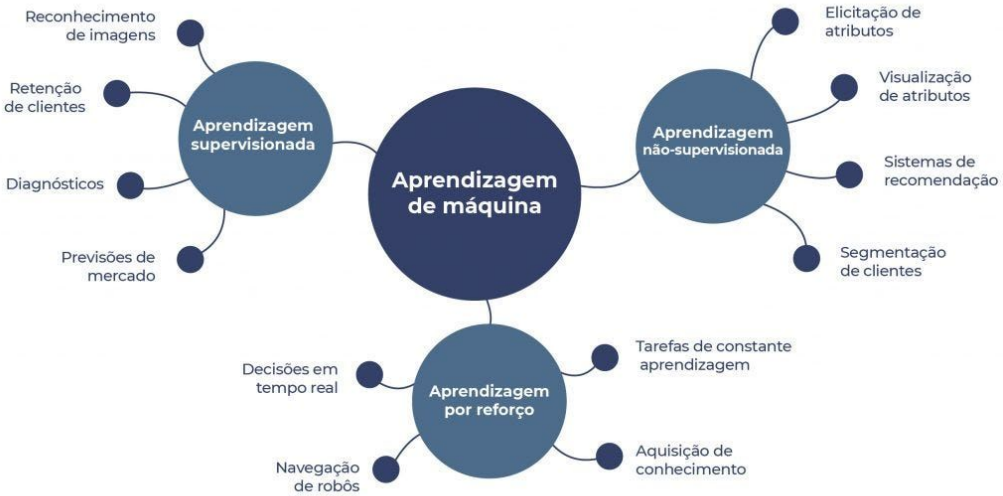
- **Decisões**
  - Alarme (manutenção preditiva, tomada de decisão em investimentos);
  - Atuação (ligar ou desligar determinada carga);
  - Recomendação (sugestão com base em padrão de consumo);
  - Indicação (área do defeito).

Aprendizado de Máquina

- **Sinais de Entrada**
  - Sensores (corrente, tensão, temperatura, umidade);
  - Áudio (forma de onda);
  - Texto (palavras, frases);
  - Séries temporais (dados medidos ao longo do tempo; preço de ações, padrão de movimentação bancária);
  - Imagens.

Aprendizado de Máquina

- Como é feito?
  - Treinamento
    - **Supervisionado (rótulos)**
    - Não Supervisionado (sem rótulos)
    - Por Reforço (feedback - tentativa e erro - recompensa)
  - Teste
  - Sintonia de parâmetros
  - Aplicação



Fonte: https://isitics.com/2018/05/10/principais-sub-divisoes-e-aplicabilidade-da-aprendizagem-de-maquina/

Aprendizado **Supervisionado**

1. **Coleta** de dados (através de sensores, pesquisas, bancos de dados existentes);
2. **Rotulação** dos dados (necessário que cada tipo de entrada já tenha uma resposta correta conhecida);
3. Após a rotulação criados dois **grupos** de dados:
    3.1. Treinamento (entre 60% e 80%);
    3.2. Teste (entre 40% e 20%);
4. Durante o **treinamento**, é realizado o **ajuste dos parâmetros** do método, com o objetivo de **reduzir o erro** na classificação;
5. Com o **modelo treinado**, qualquer nova entrada terá **pesos** atribuídos **conforme suas características**;
6. Esses pesos irão **classificar** a entrada em um dos conjuntos pré-determinados;

Aprendizado Supervisionado

Como saber se o resultado é satisfatório?

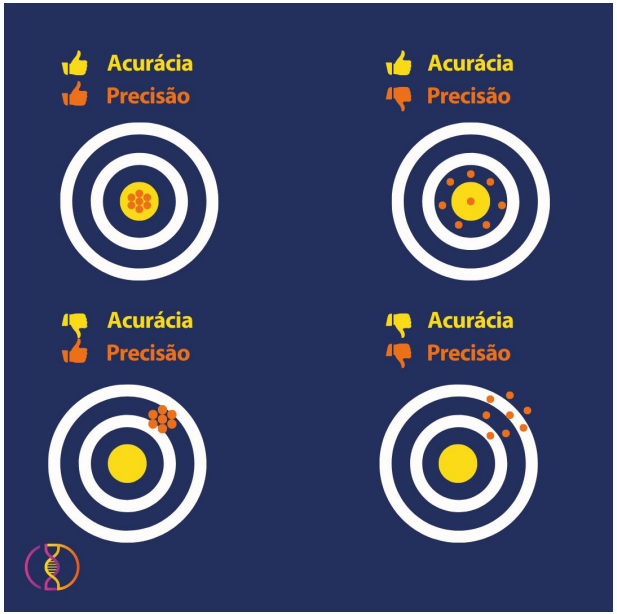
- Métricas de desempenho
- Alguns Exemplos:
  - Acurácia
  - Area under ROC Curve (Área sob a curva ROC)
  - Weighted Average Precision (Precisão Média Ponderada)
  - Weighted Average Recall (Revocação Média Ponderada)
  - Weighted Average F1 Score (F1 Score Médio Ponderado)
  - Matriz Confusão

Aprendizado Supervisionado

- **Acurácia**
  - Razão entre previsões corretas e total de amostras
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor

$$Acurácia = \frac{Previsões\ Corretas}{Total\ de\ Amostras}$$

Fonte: scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

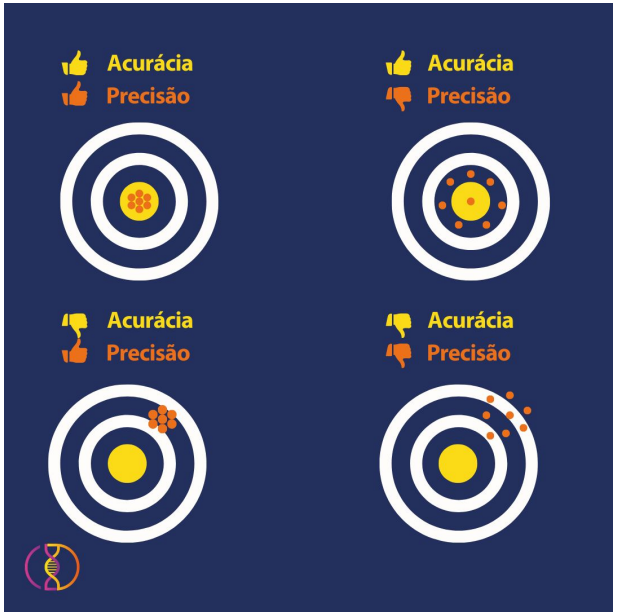


Fonte: doutoresgma.com.br/qual-a-diferenca-de-acuracia-e-precisao

Aprendizado Supervisionado

- **Precisão**
  - Razão entre Positivos Verdadeiros e total de previsões positivas
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor

$$Precisão = \frac{Positivos\ Verdadeiros}{Positivos\ Verdadeirso + Falsos\ Verdadeiros}$$



Fonte: doutoresgma.com.br/qual-a-diferenca-de-acuracia-e-precisao

Fonte: scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

Aprendizado Supervisionado

- **Area under ROC Curve (AUC - ROC)**
  - Representação visual do desempenho do modelo
  - AUC: área sob a curva
  - ROC: curva de característica de operação do receptor
  - Eixos do gráfico:
    - Taxa de verdadeiro positivo
    - Taxa de falso positivo
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor.

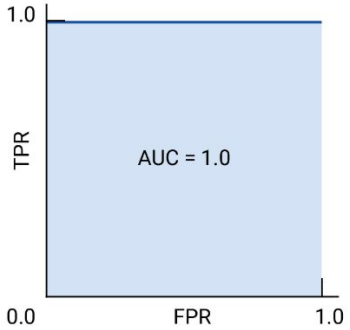


Figura 1. *ROC e AUC de um modelo hipotético perfeito.*

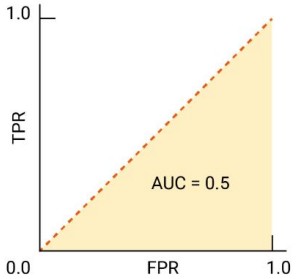


Figura 2. *ROC e AUC de palpites completamente aleatórios.*

Fonte: https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc?hl=pt-br#receiver-operating_characteristic_curve_roc

Aprendizado Supervisionado

- **Weighted Average Precision (Precisão Média Ponderada)**
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor
  - Média ponderada da precisão de cada classe
  - Considera o número de amostras

$$Precisão_{ponderada} = \frac{\sum_{i=1}^{K} Precisão_i \times n_i}{\sum_{i=1}^{K} n_i}$$

K = número de classes

Precisão_i = precisão da classe i: $\frac{TP_i}{TP_i + FP_i}$

n_i = número de amostras verdadeiras da classe i (também chamado de "suporte" da classe)

TP_i = verdadeiros positivos da classe i

FP_i = falsos positivos da classe i

Observação: métricas ponderadas fazem com que classes com mais dados não mascarem classes menores

Fonte: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

Aprendizado Supervisionado

- **Weighted Average Recall (Revocação Média Ponderada)**
  - Razão entre o número de verdadeiros positivos e o número total de casos positivos
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor
  - Valores abaixo de 0,7 são considerados ruins
  - Avaliação do desempenho global de um classificador

$$\text{Recall}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}$$

TP = Verdadeiros Positivos (acertos da classe)

FN = Falsos Negativos (erros em que a classe foi ignorada)

$$\text{Recall}_{\text{weighted}} = \frac{\sum_{i=1}^{K} \text{Recall}_i \times n_i}{\sum_{i=1}^{K} n_i}$$

K = número total de classes

n_i = número de amostras reais da classe i

Fonte: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

Aprendizado Supervisionado

- **Weighted Average F1 Score (F1 Score Médio Ponderado)**
  - Valores entre 0 e 1
  - Quanto mais próximo de 1, melhor
  - Valores abaixo de 0,7 são considerados ruins
  - Avaliação do desempenho global de um classificador

$$F1_i = 2 \cdot \frac{Precisão_i \cdot Recall_i}{Precisão_i + Recall_i}$$

$$F1_{ponderado} = \sum \omega_i \cdot F1_i$$

$$\omega_i = \frac{Qde\ de\ amostras\ da\ classe}{Total\ de\ amostras}$$

Fonte: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

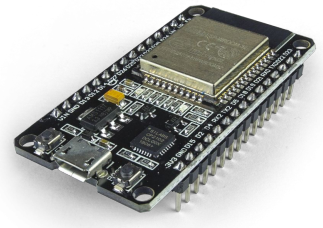
Alguns métodos de aprendizado

- **Redes Neurais Artificiais (ANN)**
- **Random Forest**
- **K-Nearest Neighbors (KNN)**
- **Suport Vector Machine (SVM)**
- **Convolutional Neural Networks (CNN)**

Atividade:

- **Uso da plataforma Edge Impulse para realizar a classificação do tipo de partida de motores elétricos**

“Gerador de Sinais” + EDGE IMPULSE +

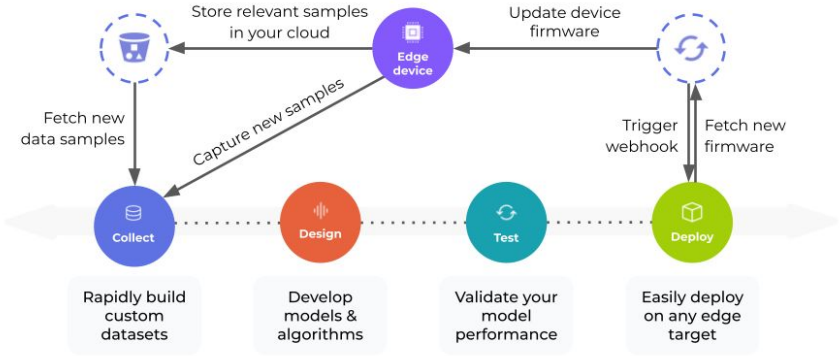


Fonte: studio.edgeimpulse.com/

Fonte: robocore.net/wifi/esp32-wifi-bluetooth

Edge Impulse

- Plataforma de IA;
- Coleta de dados;
- Treinamento de modelos;
- Implantação de modelos em dispositivos de computação;
- Interface web;
- Recursos gratuitos;
- Alternativa para aprendizado e soluções de mercado.



Fonte: docs.edgeimpulse.com/docs/concepts/edge-ai-fundamentals/what-is-edge-impulse

Conjunto de dados utilizado

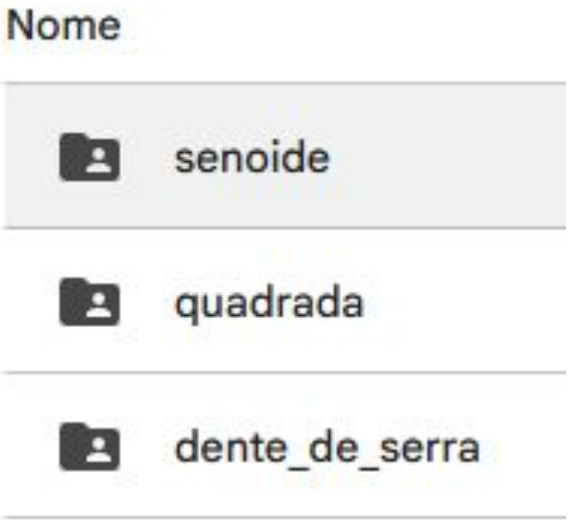
- Gerados **Artificialmente** em python
- Simulações de **formas de onda**:
  - Senoidal
  - Quadrada
  - Dente de Serra

Link para o conjunto de dados



drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado



Link para o conjunto de dados

drive.google.com/drive/folders/1g7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado

- Senoide

```python
def gerar_senoide():
    freq = np.random.uniform(0.5, 5)
    fase = np.random.uniform(0, 2*np.pi)
    t = np.linspace(0, 1, num_pontos)
    return np.sin(2 * np.pi * freq * t + fase)
```

Link para o conjunto de dados

drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado

- Quadrada

```
def gerar_quadrada():
    freq = np.random.uniform(0.5, 5)
    t = np.linspace(0, 1, num_pontos)
    return np.sign(np.sin(2 * np.pi * freq * t))
```

Link para o conjunto de dados



drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado

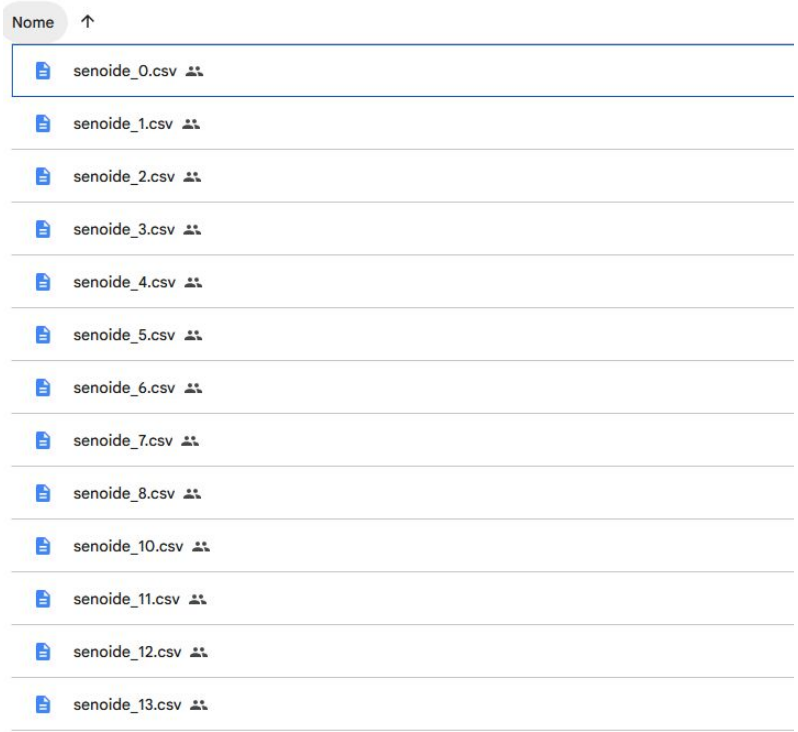
- Dente de Serra

```python
def gerar_dente_de_serra():
    freq = np.random.uniform(0.5, 5)
    t = np.linspace(0, 1, num_pontos)
    return 2 * (t * freq % 1) - 1
```

Link para o conjunto de dados

drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado



Link para o conjunto de dados

drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Conjunto de dados utilizado

senoide_0.csv

	A	B
1	timestamp	value
2	0	1.52759662
3	10	1.858153243
4	20	1.98323379
5	30	2.075253812
6	40	2.270191579
7	50	2.289915595

```python
# Parâmetros
num_amostras = 1000
num_pontos = 100
max_tensao = 3.3
pastas = ['senoide', 'quadrada', 'dente_de_serra', 'ruido']

# Subpastas
for d in pastas:
    os.makedirs(d, exist_ok=True)

# Vetor de tempo de 0 a 1000 ms, com num_pontos pontos
timestamps = np.linspace(0, 1000, num_pontos, dtype=int)
```

Link para o conjunto de dados

drive.google.com/drive/folders/1q7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

Plataforma Edge Impulse

Acessar o site e criar uma conta:

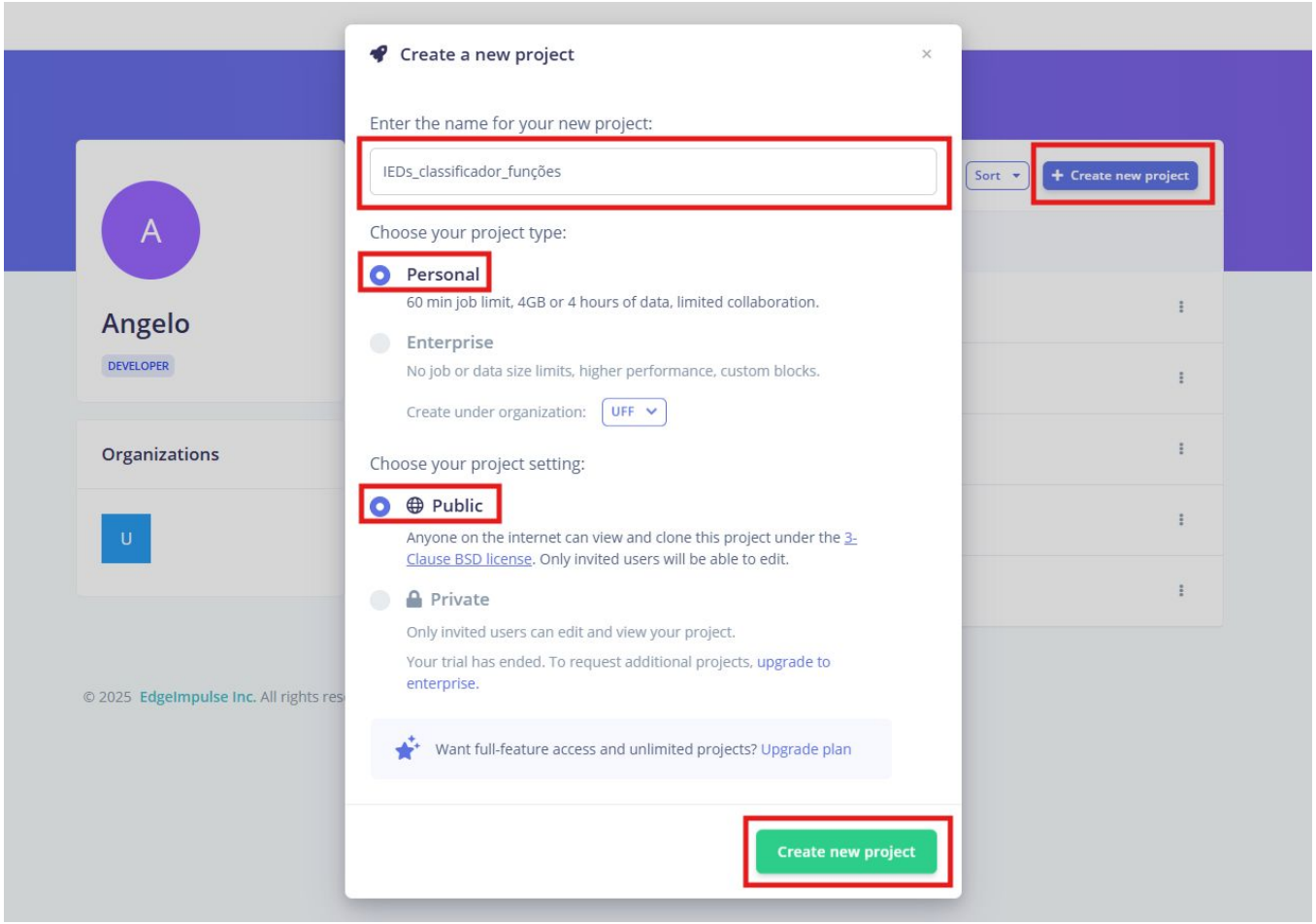
studio.edgeimpulse.com

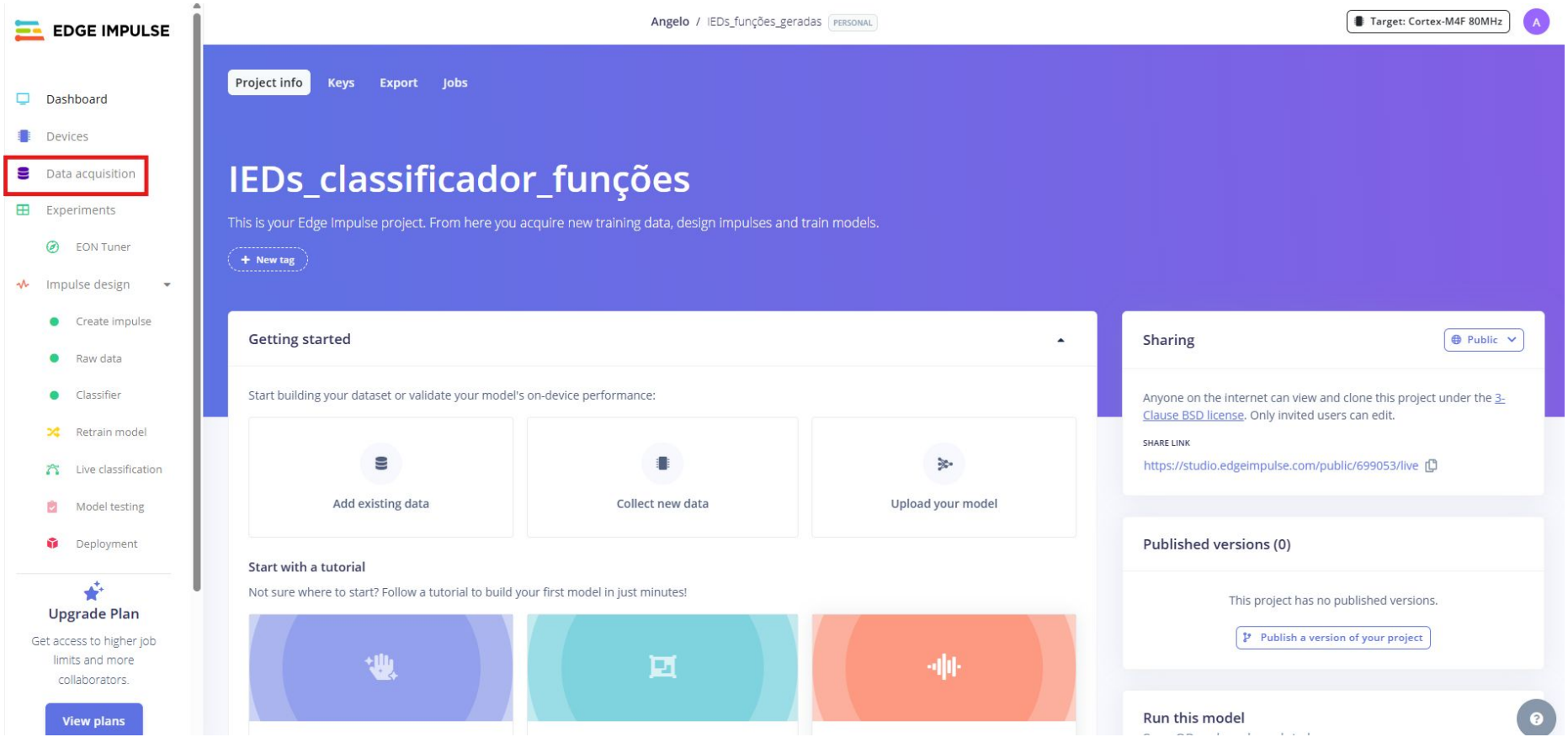
Link para o conjunto de dados

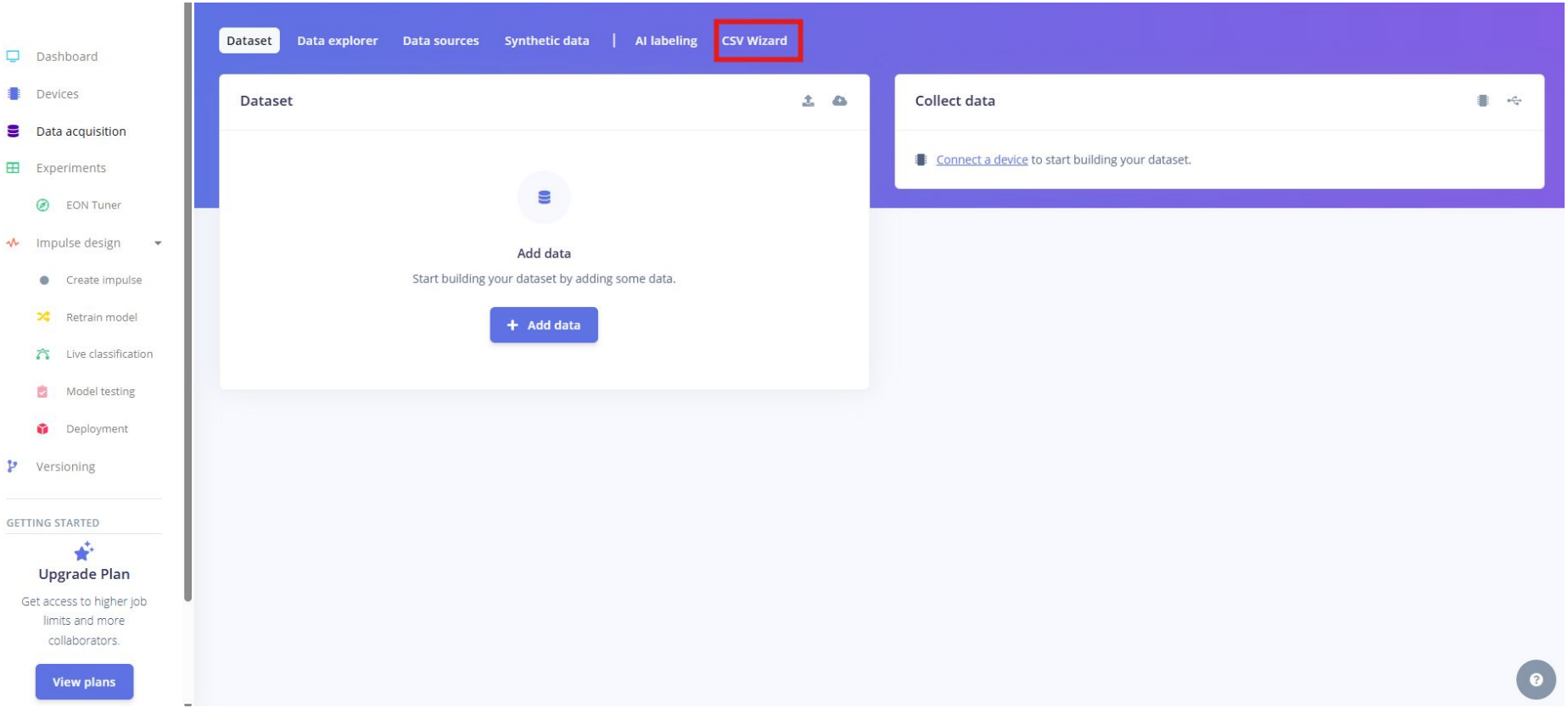
drive.google.com/drive/folders/1g7L3WsyK2oesfC6hus6eSU6Mds64luFQ?usp=drive_link

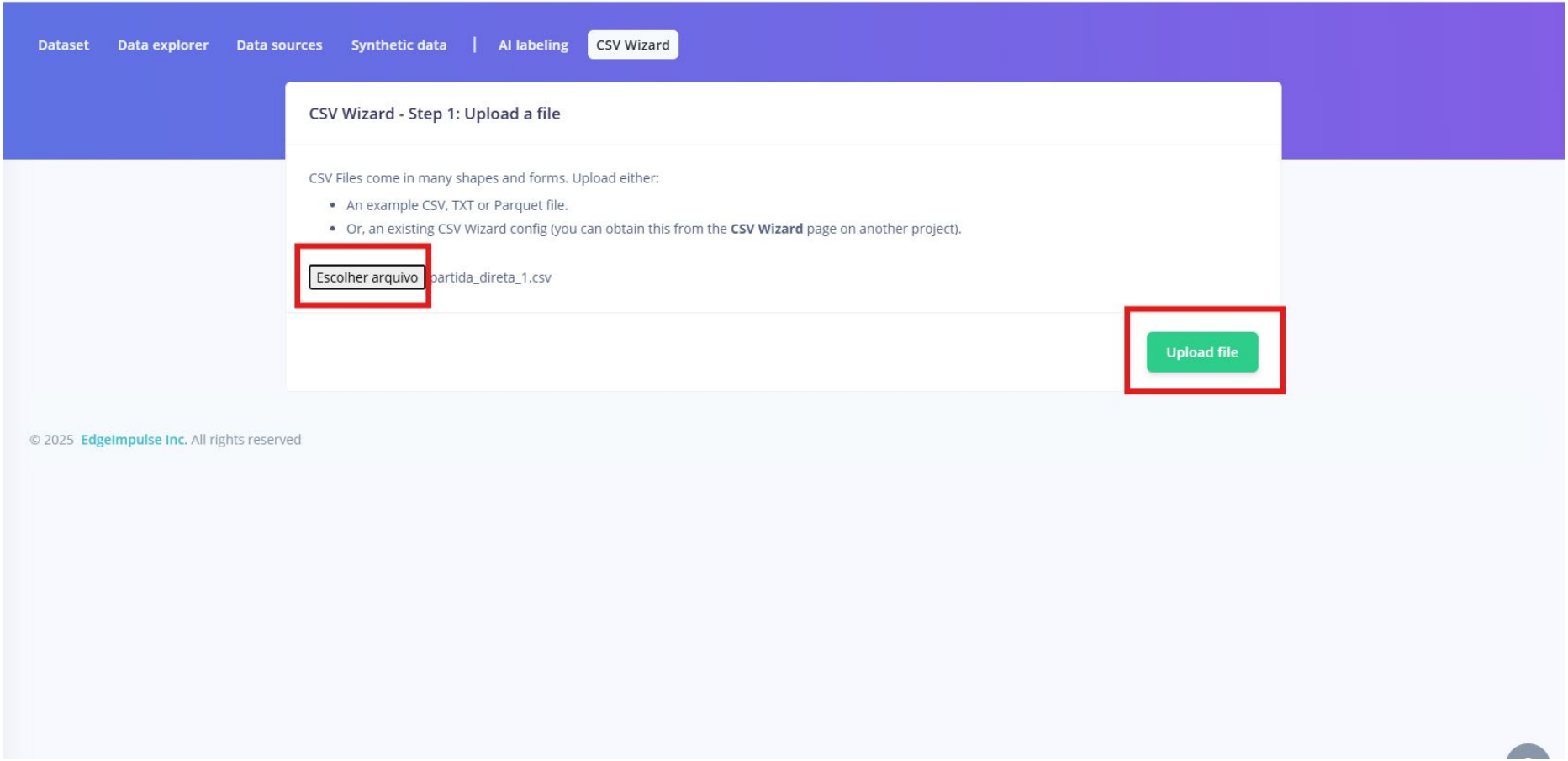
Plataforma Edge Impulse

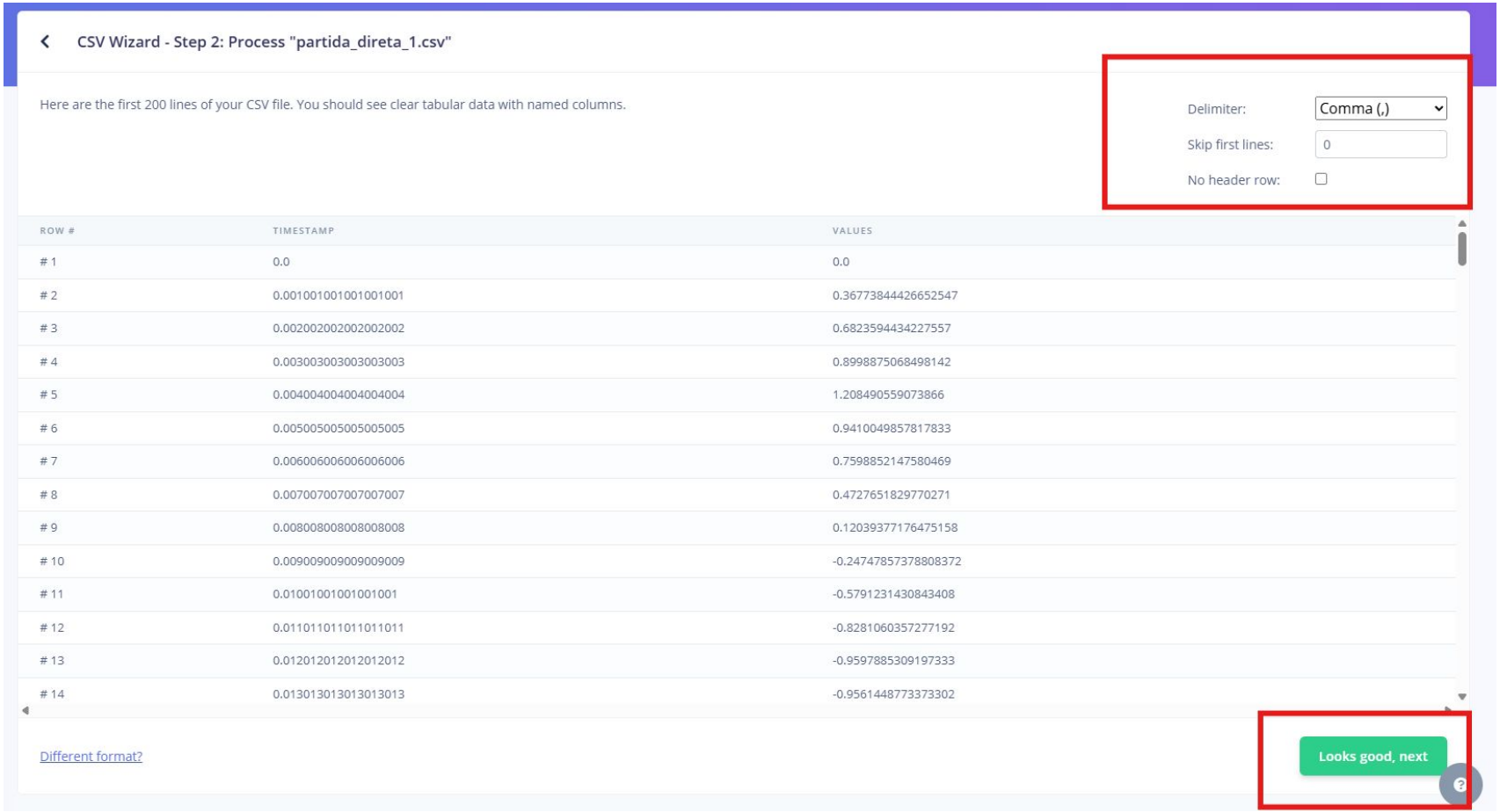
Criando e Treinando o Modelo

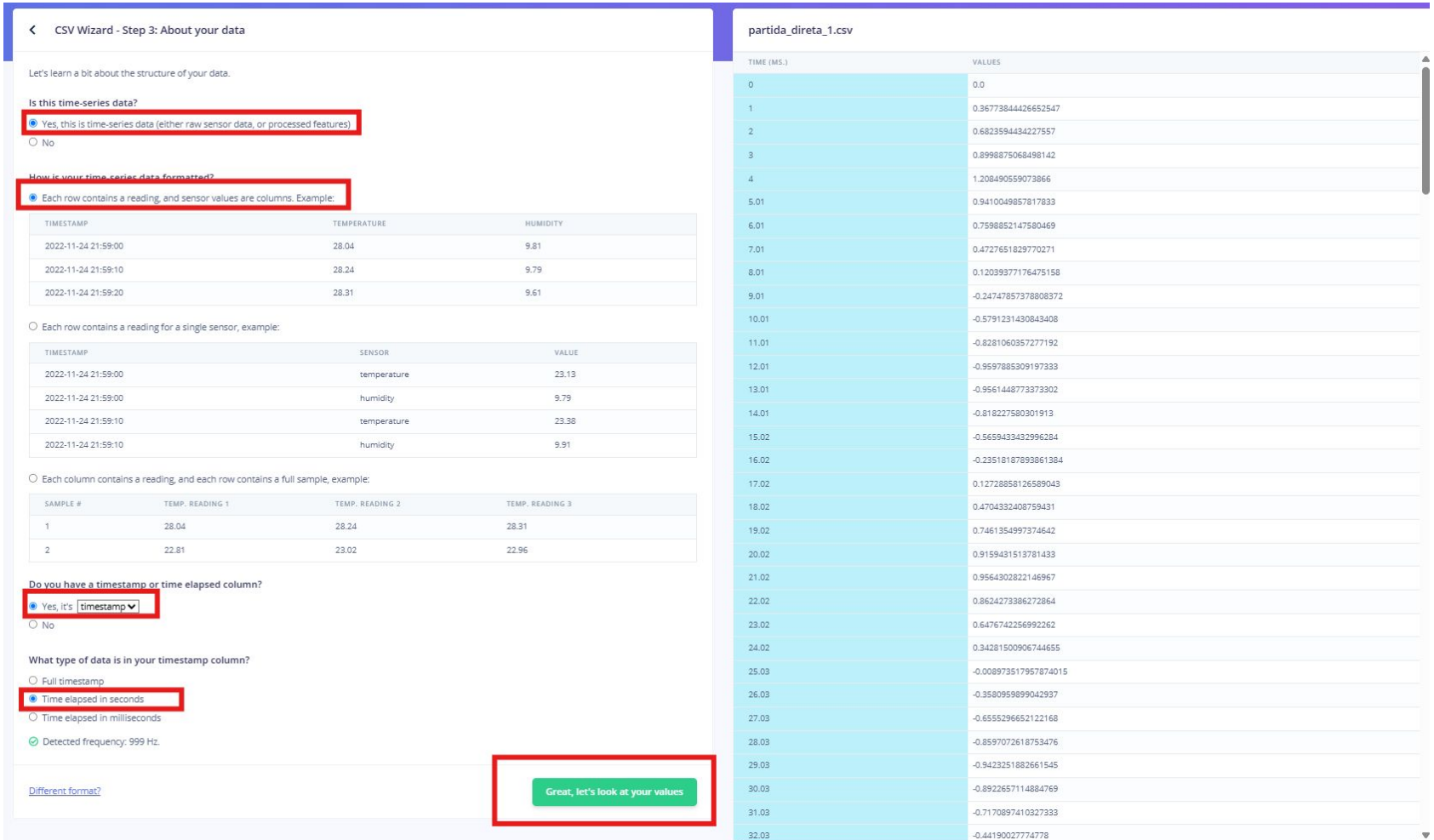

Create a new project
Enter the name for your new project:
IEDs_classificador_funções
Choose your project type:
Personal
60 min job limit, 4GB or 4 hours of data, limited collaboration.
Enterprise
No job or data size limits, higher performance, custom blocks.
Create under organization:
UFF
Choose your project setting:
Public
Anyone on the internet can view and clone this project under the 3-Clause BSD license. Only invited users will be able to edit.
Private
Only invited users can edit and view your project.
Your trial has ended. To request additional projects, upgrade to enterprise.
Want full-feature access and unlimited projects? Upgrade plan
Create new project
Sort
+ Create new project
A
Angelo
DEVELOPER
Organizations
U
© 2025 EdgeImpulse Inc. All rights res


EDGE IMPULSE
Angelo / IEDs_funções_geradas
PERSONAL
Target: Cortex-M4F 80MHz
A
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Project info
Keys
Export
Jobs
IEDs_classificador_funções
This is your Edge Impulse project. From here you acquire new training data, design impulses and train models.
+ New tag
Getting started
Start building your dataset or validate your model's on-device performance:
Add existing data
Collect new data
Upload your model
Start with a tutorial
Not sure where to start? Follow a tutorial to build your first model in just minutes!
Sharing
Public
Anyone on the internet can view and clone this project under the 3-Clause BSD license. Only invited users can edit.
SHARE LINK
https://studio.edgeimpulse.com/public/699053/live
Published versions (0)
This project has no published versions.
Publish a version of your project
Run this model


Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Dataset
Data explorer
Data sources
Synthetic data
AI labeling
CSV Wizard
Dataset
Add data
Start building your dataset by adding some data.
+ Add data
Collect data
Connect a device to start building your dataset.


Dataset
Data explorer
Data sources
Synthetic data
AI labeling
CSV Wizard
CSV Wizard - Step 1: Upload a file
CSV Files come in many shapes and forms. Upload either:
An example CSV, TXT or Parquet file.
Or, an existing CSV Wizard config (you can obtain this from the CSV Wizard page on another project).
Escolher arquivo
partida_direta_1.csv
Upload file
© 2025 EdgeImpulse Inc. All rights reserved


CSV Wizard - Step 2: Process "partida_direta_1.csv"
Here are the first 200 lines of your CSV file. You should see clear tabular data with named columns.
Delimiter:
Comma (,)
Skip first lines:
0
No header row:
ROW #
TIMESTAMP
VALUES
1
0.0
0.0
2
0.001001001001001001
0.36773844426652547
3
0.002002002002002002
0.6823594434227557
4
0.003003003003003003
0.8998875068498142
5
0.004004004004004004
1.208490559073866
6
0.005005005005005005
0.9410049857817833
7
0.006006006006006006
0.7598852147580469
8
0.007007007007007007
0.4727651829770271
9
0.008008008008008008
0.12039377176475158
10
0.009009009009009009
-0.24747857378808372
11
0.01001001001001001
-0.5791231430843408
12
0.011011011011011011
-0.8281060357277192
13
0.012012012012012012
-0.9597885309197333
14
0.013013013013013013
-0.9561448773373302
Different format?
Looks good, next


CSV Wizard - Step 3: About your data
Let's learn a bit about the structure of your data.
Is this time-series data?
Yes, this is time-series data (either raw sensor data, or processed features)
No
How is your time-series data formatted?
Each row contains a reading, and sensor values are columns. Example:
TIMESTAMP TEMPERATURE HUMIDITY
2022-11-24 21:59:00 28.04 9.81
2022-11-24 21:59:10 28.24 9.79
2022-11-24 21:59:20 28.31 9.61
Each row contains a reading for a single sensor, example:
TIMESTAMP SENSOR VALUE
2022-11-24 21:59:00 temperature 23.13
2022-11-24 21:59:00 humidity 9.79
2022-11-24 21:59:10 temperature 23.38
2022-11-24 21:59:10 humidity 9.91
Each column contains a reading, and each row contains a full sample, example:
SAMPLE # TEMP. READING 1 TEMP. READING 2 TEMP. READING 3
1 28.04 28.24 28.31
2 22.81 23.02 22.96
Do you have a timestamp or time elapsed column?
Yes, it's timestamp
No
What type of data is in your timestamp column?
Full timestamp
Time elapsed in seconds
Time elapsed in milliseconds
Detected frequency: 999 Hz.
Different format?
Great, let's look at your values
partida_direta_1.csv
TIME (MS.) VALUES
0 0.0
1 0.36773844426652547
2 0.6823594434227557
3 0.8998875068498142
4 1.208490559073866
5.01 0.9410049857817833
6.01 0.7598852147580469
7.01 0.4727651829770271
8.01 0.12039377176475158
9.01 -0.24747857378808372
10.01 -0.5791231430843408
11.01 -0.8281060357277192
12.01 -0.9597885309197333
13.01 -0.9561448773373302
14.01 -0.818227580301913
15.02 -0.5659433432996284
16.02 -0.23518187893861384
17.02 0.12728858126589043
18.02 0.4704332408759431
19.02 0.7461354997374642
20.02 0.9159431513781433
21.02 0.9564302822146967
22.02 0.8624273386272864
23.02 0.6476742256992262
24.02 0.34281500906744655
25.03 -0.008973517957874015
26.03 -0.3580959899042937
27.03 -0.6555296652122168
28.03 -0.8597072618753476
29.03 -0.9423251882661545
30.03 -0.8922657114884769
31.03 -0.7170897410327333
32.03 -0.44190027774778

CSV Wizard - Step 4: About your values

Almost there. Let's parse your values.

Do you have a column that contains the label (the value you want to predict)?

- ○ Yes, it's values
- ◉ No (no worries, you can provide this when you upload data)

Which columns contain your values?

- ☑ values

Different format?

Next, split up into samples

Example sample

TIME (MS.)	VALUES
0	0.0
1	0.36773844426652547
2	0.6823594434227557
3	0.8998875068498142
4	1.208490559073866
5.01	0.9410049857817833
6.01	0.7598852147580469
7.01	0.4727651829770271
8.01	0.12039377176475158

Sample graph



CSV Wizard - Step 5: Split up your samples

If you have a lot of data we can automatically split one CSV file up in multiple samples.

How long do you want your samples to be?

- (•) Unlimited
- ( ) Limit to _____ ms.

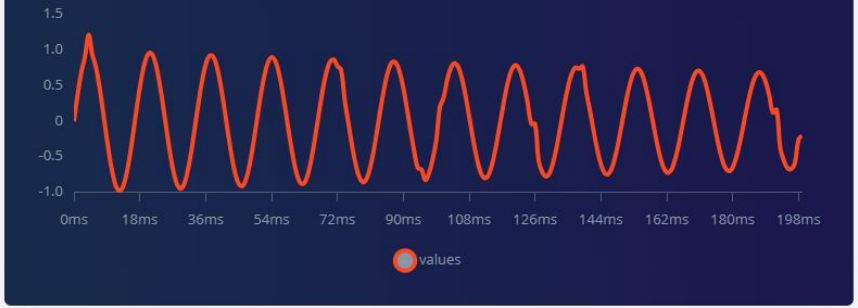
Different format?

Finish wizard

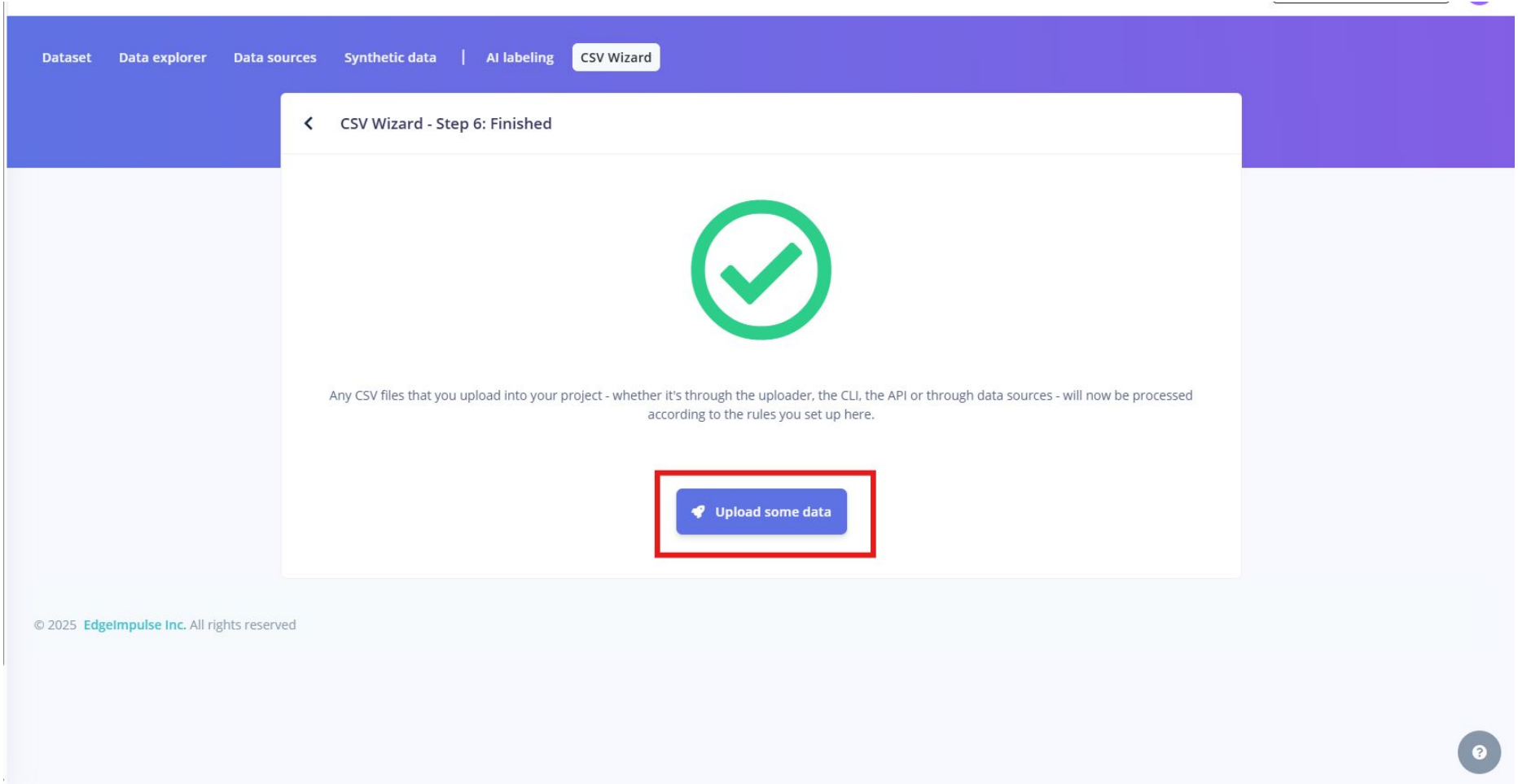
Example sample

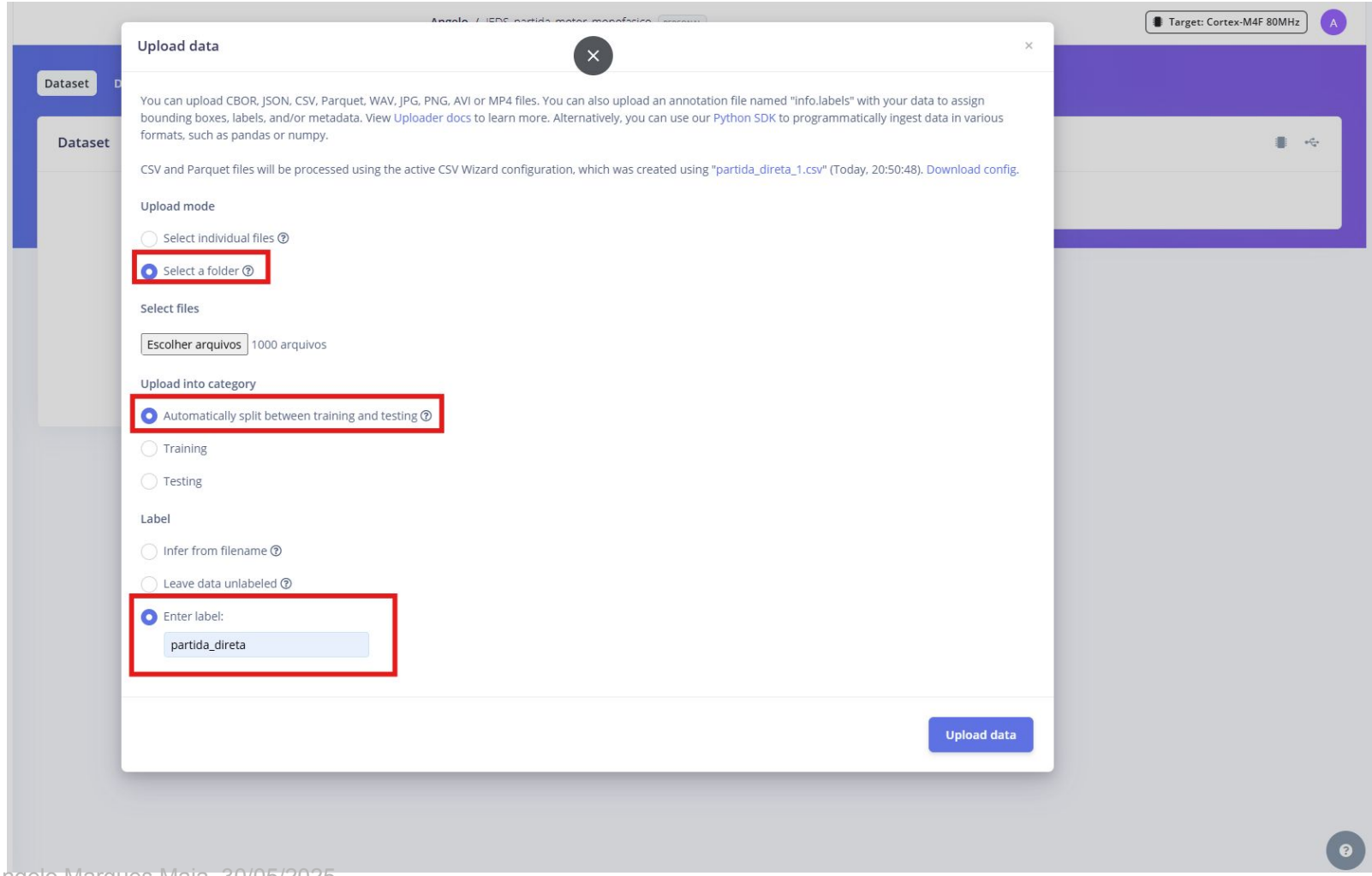
TIME (MS.)	VALUES
Sample 1	
0	0.0
1	0.36773844426652547
2	0.6823594434227557
3	0.8998875068498142
4	1.208490559073866
5.01	0.9410049857817833
6.01	0.7598852147580469
7.01	0.4727651829770271

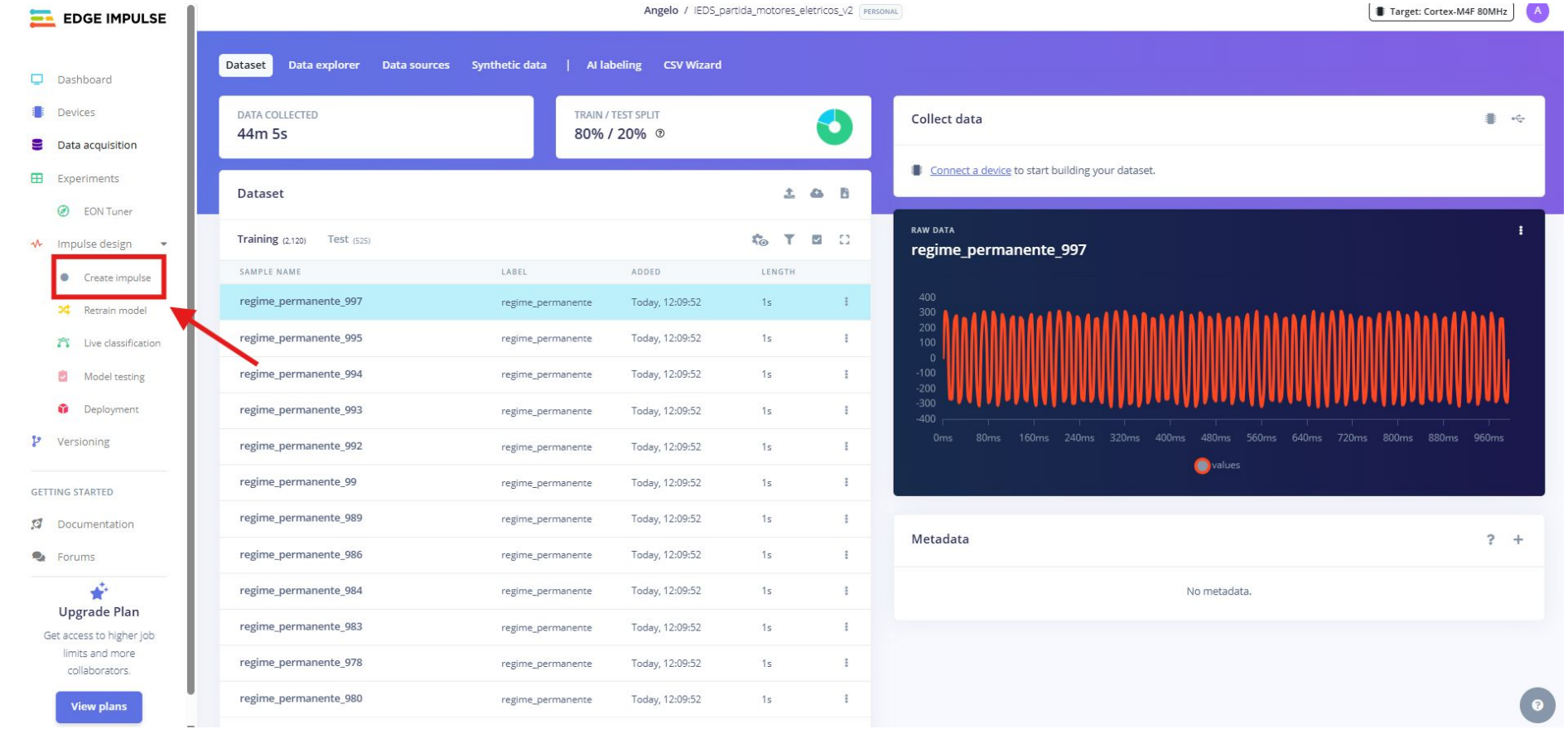
Sample graph

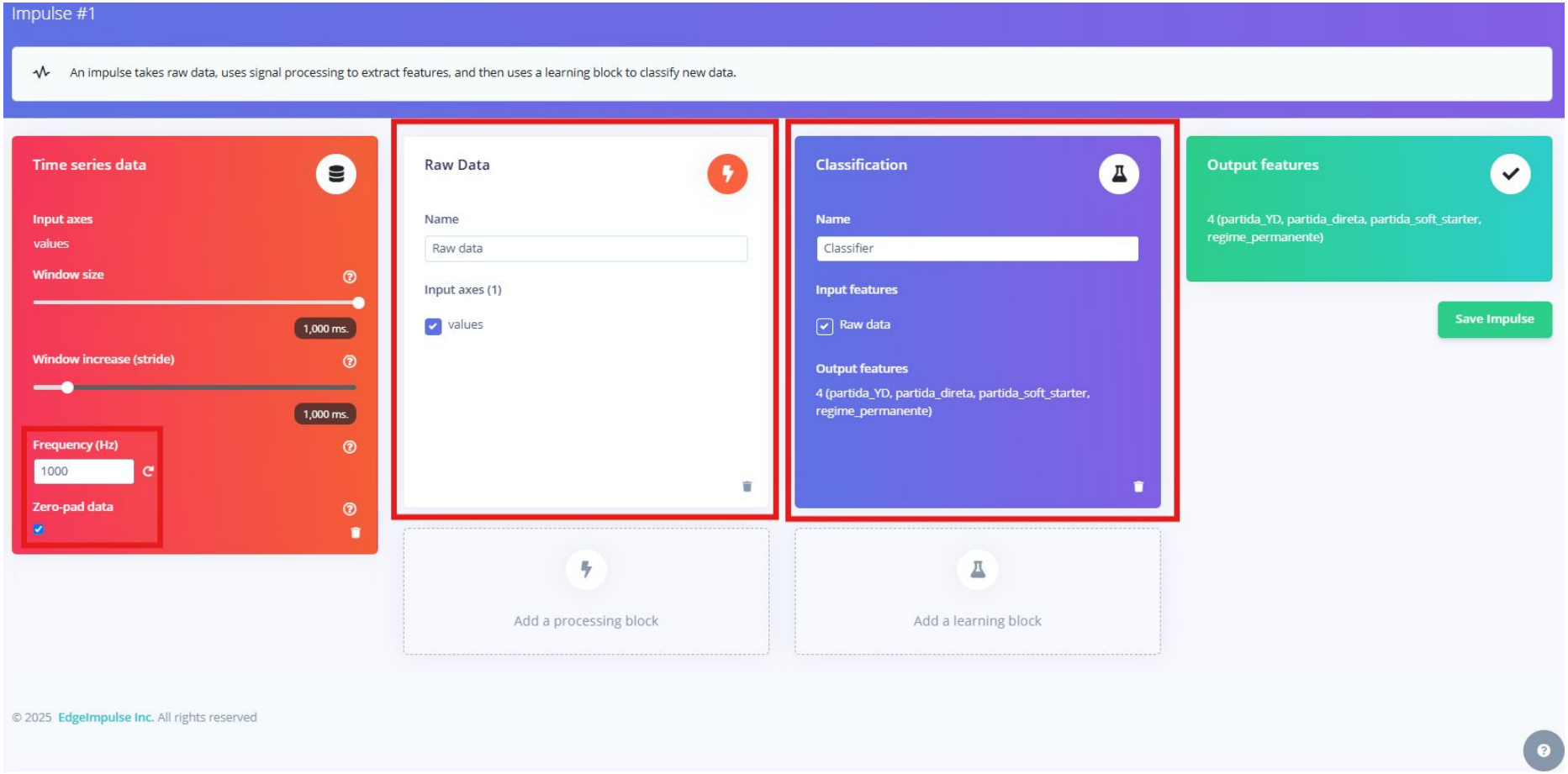


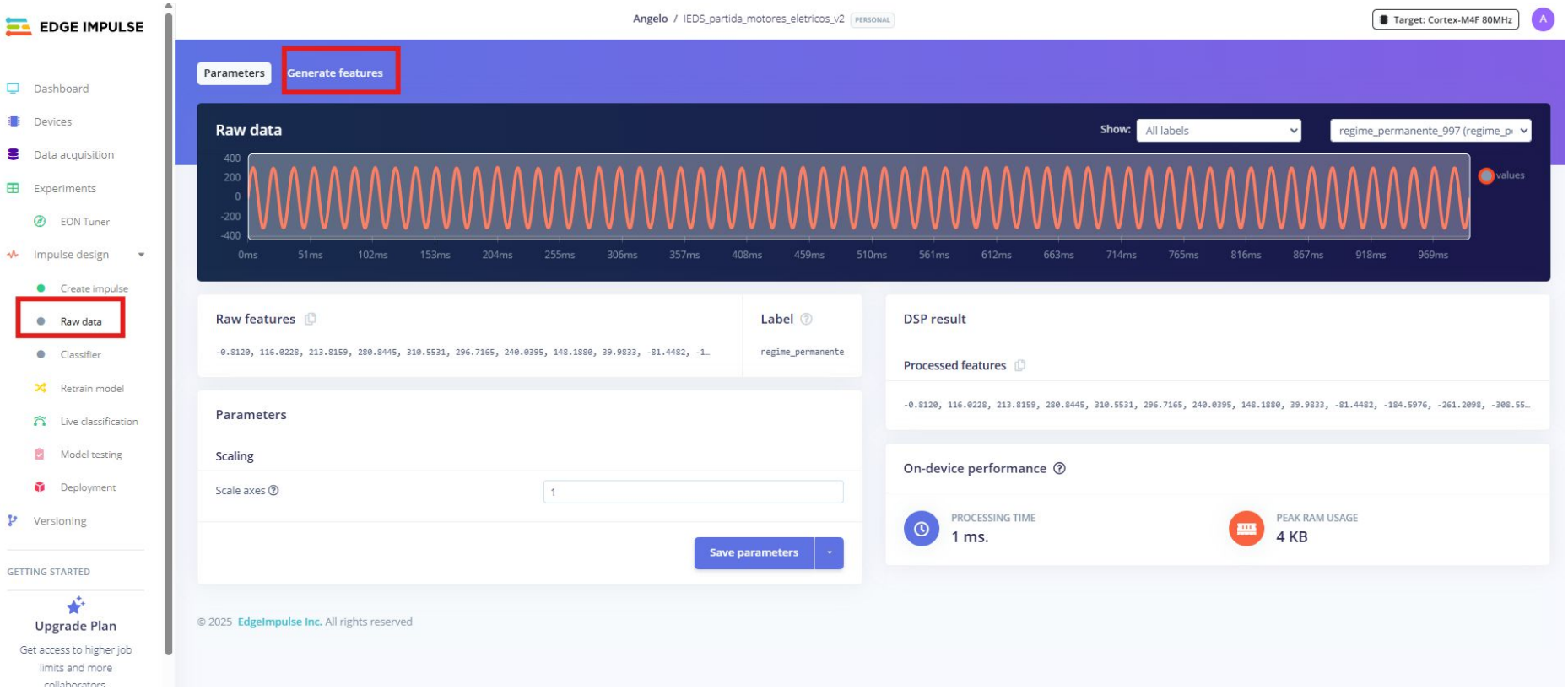
© 2025 EdgeImpulse Inc. All rights reserved

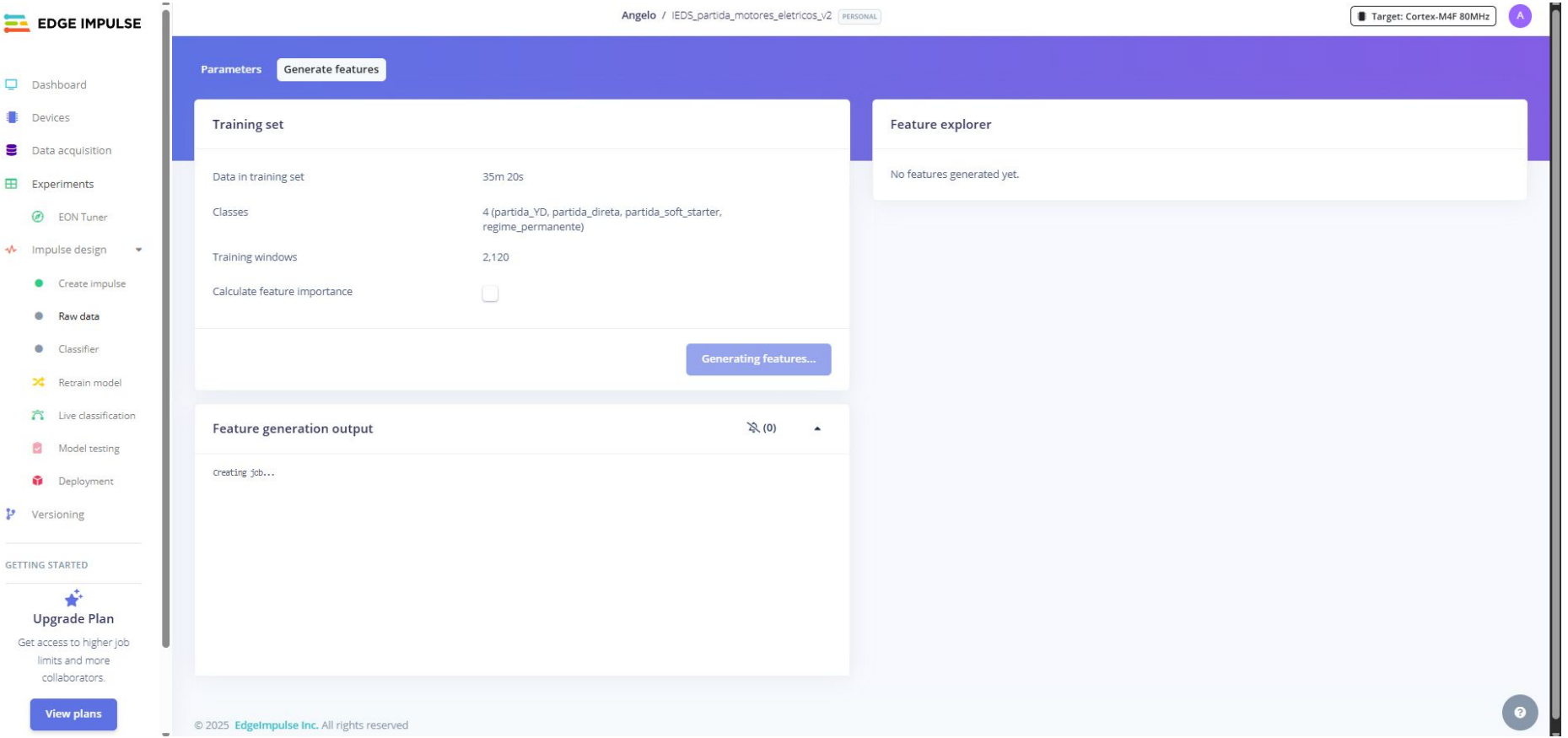

Dataset
Data explorer
Data sources
Synthetic data
AI labeling
CSV Wizard
CSV Wizard - Step 6: Finished
Any CSV files that you upload into your project - whether it's through the uploader, the CLI, the API or through data sources - will now be processed according to the rules you set up here.
Upload some data
© 2025 EdgeImpulse Inc. All rights reserved

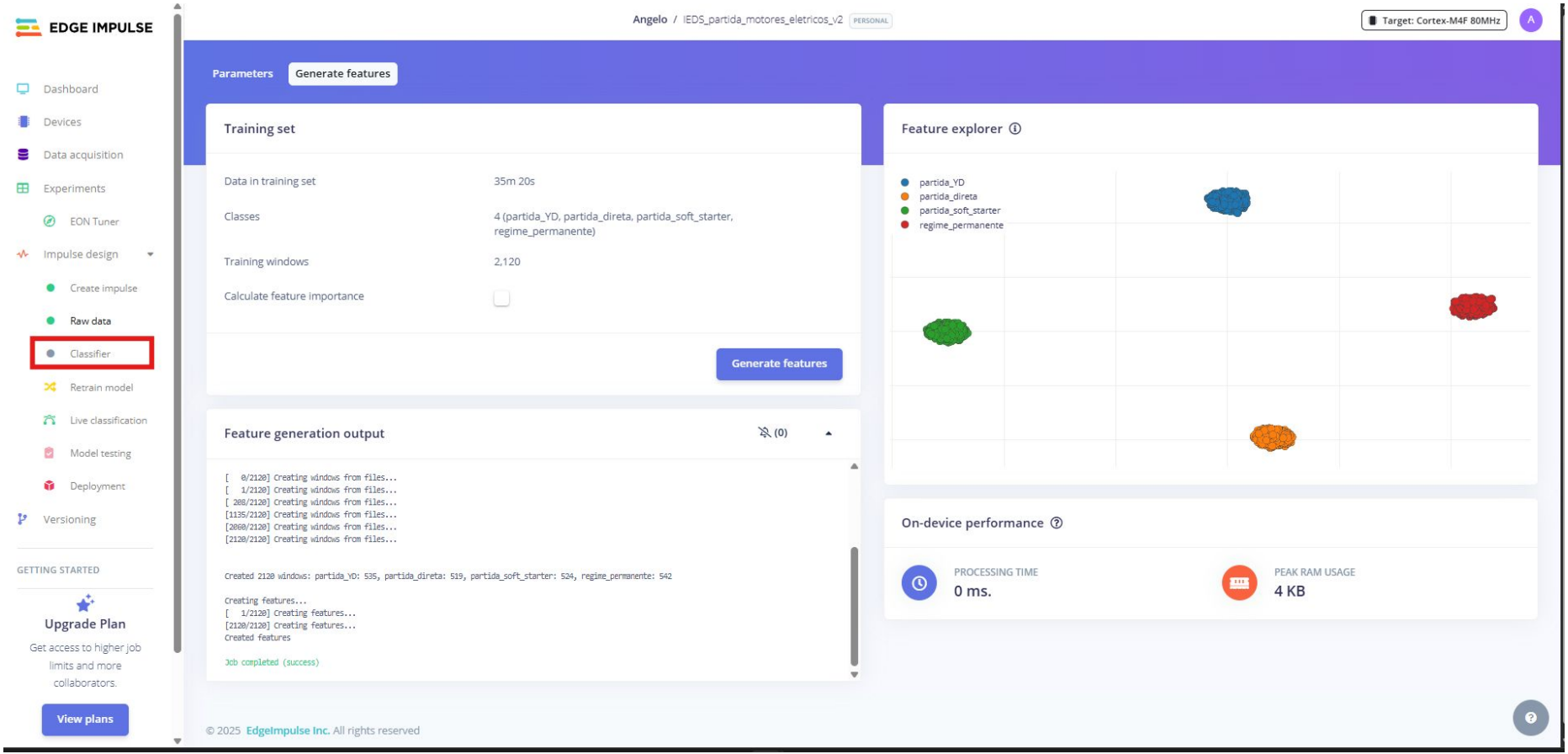

Target: Cortex-M4F 80MHz
A
Dataset
Dataset
Upload data
You can upload CBOR, JSON, CSV, Parquet, WAV, JPG, PNG, AVI or MP4 files. You can also upload an annotation file named "info.labels" with your data to assign bounding boxes, labels, and/or metadata. View Uploader docs to learn more. Alternatively, you can use our Python SDK to programmatically ingest data in various formats, such as pandas or numpy.
CSV and Parquet files will be processed using the active CSV Wizard configuration, which was created using "partida_direta_1.csv" (Today, 20:50:48). Download config.
Upload mode
Select individual files
Select a folder
Select files
Escolher arquivos
1000 arquivos
Upload into category
Automatically split between training and testing
Training
Testing
Label
Infer from filename
Leave data unlabeled
Enter label:
partida_direta
Upload data

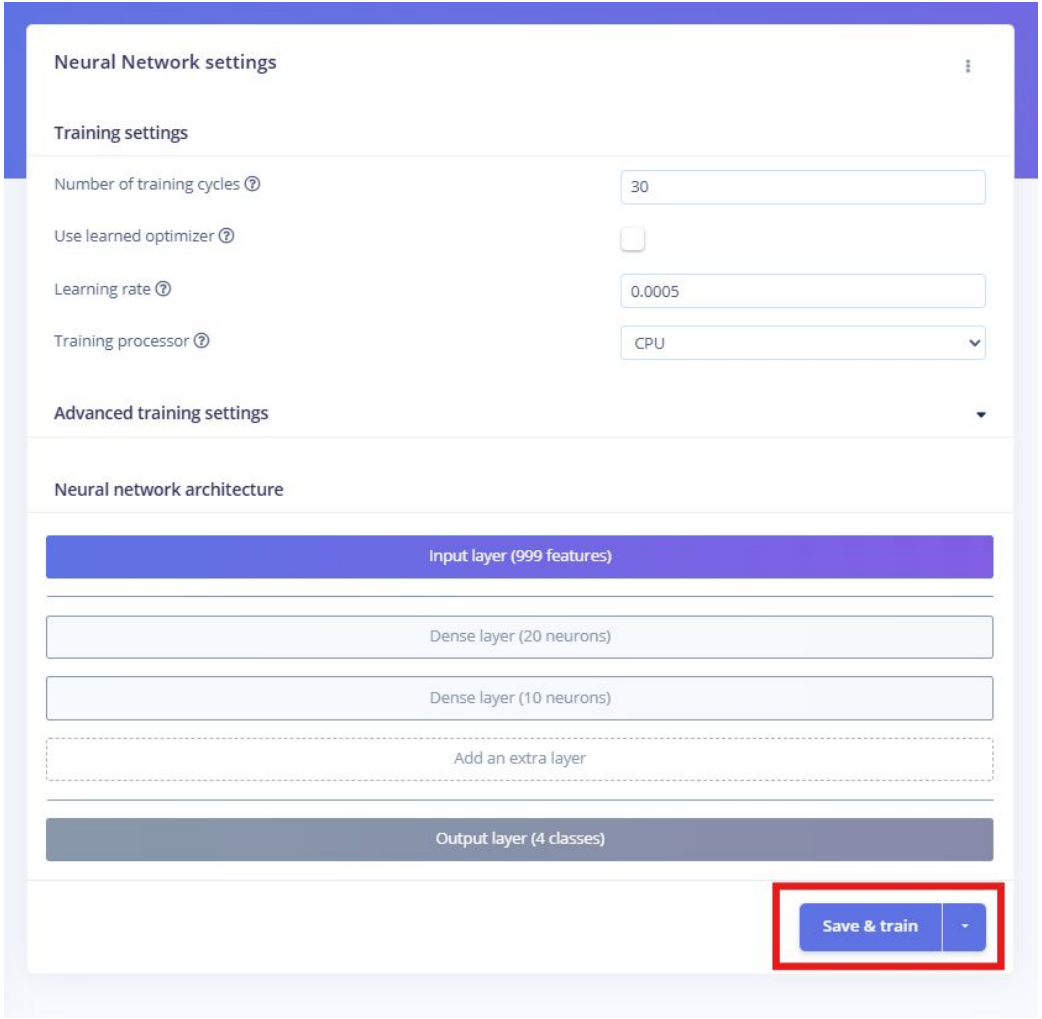

EDGE IMPULSE
Angelo / IEDS_partida_motores_eletricos_v2
PERSONAL
Target: Cortex-M4F 80MHz
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Documentation
Forums
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Dataset
Data explorer
Data sources
Synthetic data
AI labeling
CSV Wizard
DATA COLLECTED
44m 5s
TRAIN / TEST SPLIT
80% / 20%
Collect data
Connect a device to start building your dataset.
Dataset
Training (2,120)
Test (525)
SAMPLE NAME
LABEL
ADDED
LENGTH
regime_permanente_997 regime_permanente Today, 12:09:52 1s
regime_permanente_995 regime_permanente Today, 12:09:52 1s
regime_permanente_994 regime_permanente Today, 12:09:52 1s
regime_permanente_993 regime_permanente Today, 12:09:52 1s
regime_permanente_992 regime_permanente Today, 12:09:52 1s
regime_permanente_99 regime_permanente Today, 12:09:52 1s
regime_permanente_989 regime_permanente Today, 12:09:52 1s
regime_permanente_986 regime_permanente Today, 12:09:52 1s
regime_permanente_984 regime_permanente Today, 12:09:52 1s
regime_permanente_983 regime_permanente Today, 12:09:52 1s
regime_permanente_978 regime_permanente Today, 12:09:52 1s
regime_permanente_980 regime_permanente Today, 12:09:52 1s
RAW DATA
regime_permanente_997
400
300
200
100
0
-100
-200
-300
-400
0ms 80ms 160ms 240ms 320ms 400ms 480ms 560ms 640ms 720ms 800ms 880ms 960ms
values
Metadata
No metadata.


Impulse #1
An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data.
Time series data
Input axes
values
Window size
1,000 ms.
Window increase (stride)
1,000 ms.
Frequency (Hz)
1000
Zero-pad data
Raw Data
Name
Raw data
Input axes (1)
values
Classification
Name
Classifier
Input features
Raw data
Output features
4 (partida_YD, partida_direta, partida_soft_starter, regime_permanente)
Output features
4 (partida_YD, partida_direta, partida_soft_starter, regime_permanente)
Save Impulse
Add a processing block
Add a learning block
© 2025 EdgeImpulse Inc. All rights reserved


EDGE IMPULSE
Angelo / IEDS_partida_motores_eletricos_v2
PERSONAL
Target: Cortex-M4F 80MHz
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators
Parameters
Generate features
Raw data
Show:
All labels
regime_permanente_997 (regime_p
values
400
200
0
-200
-400
0ms
51ms
102ms
153ms
204ms
255ms
306ms
357ms
408ms
459ms
510ms
561ms
612ms
663ms
714ms
765ms
816ms
867ms
918ms
969ms
Raw features
-0.8120, 116.0228, 213.8159, 280.8445, 310.5531, 296.7165, 240.0395, 148.1880, 39.9833, -81.4482, -1…
Label
regime_permanente
DSP result
Processed features
-0.8120, 116.0228, 213.8159, 280.8445, 310.5531, 296.7165, 240.0395, 148.1880, 39.9833, -81.4482, -184.5976, -261.2098, -308.55…
Parameters
Scaling
Scale axes
1
Save parameters
On-device performance
PROCESSING TIME
1 ms.
PEAK RAM USAGE
4 KB
© 2025 EdgeImpulse Inc. All rights reserved


EDGE IMPULSE
Angelo / IEDS_partida_motores_eletricos_v2
PERSONAL
Target: Cortex-M4F 80MHz
A
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Parameters
Generate features
Training set
Data in training set
35m 20s
Classes
4 (partida_YD, partida_direta, partida_soft_starter, regime_permanente)
Training windows
2,120
Calculate feature importance
Generating features...
Feature explorer
No features generated yet.
Feature generation output
(0)
Creating job...
© 2025 EdgeImpulse Inc. All rights reserved

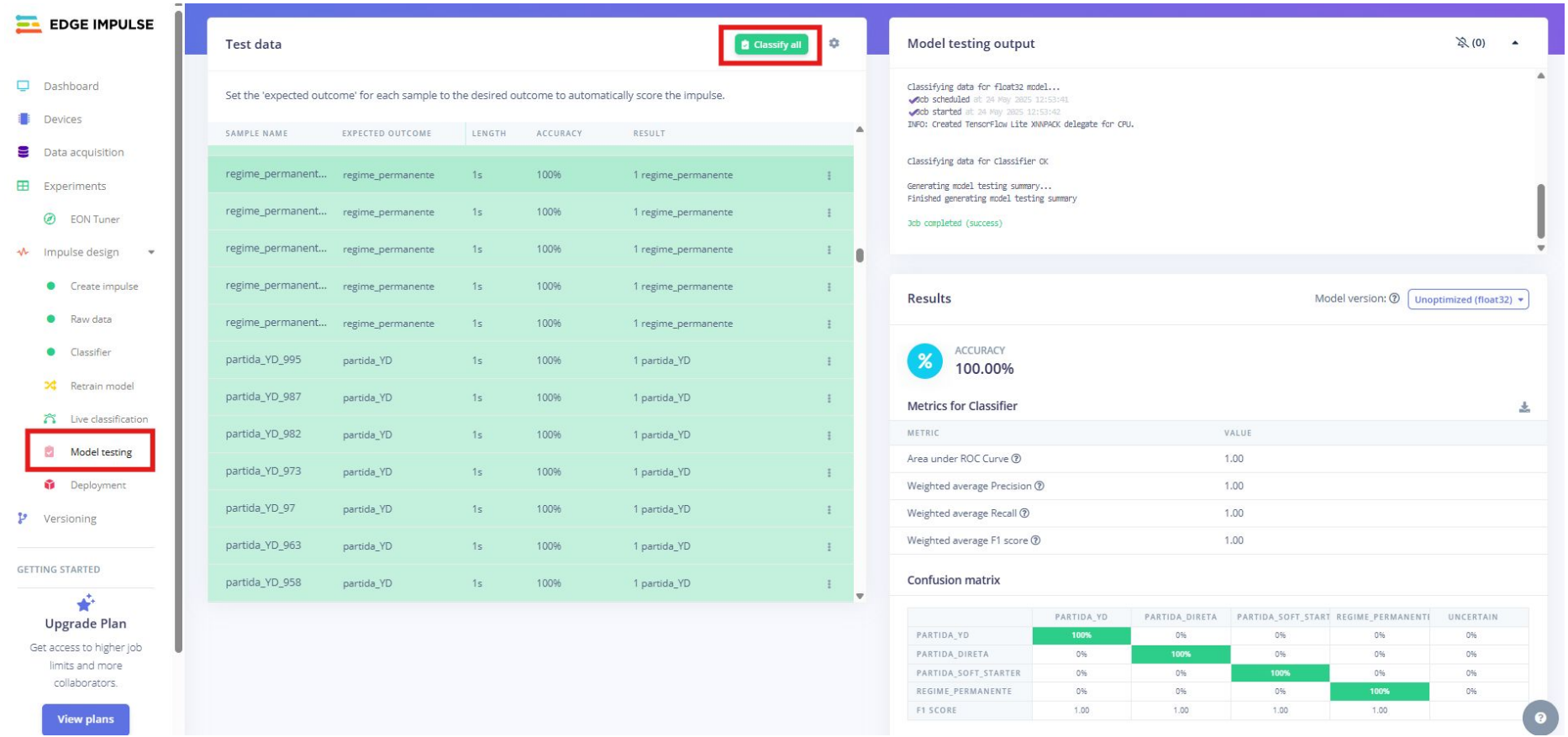

EDGE IMPULSE
Angelo / IEDS_partida_motores_eletricos_v2
PERSONAL
Target: Cortex-M4F 80MHz
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Parameters
Generate features
Training set
Data in training set
35m 20s
Classes
4 (partida_YD, partida_direta, partida_soft_starter, regime_permanente)
Training windows
2,120
Calculate feature importance
Generate features
Feature generation output
(0)
[0/2120] Creating windows from files...
[1/2120] Creating windows from files...
[208/2120] Creating windows from files...
[1135/2120] Creating windows from files...
[2080/2120] Creating windows from files...
[2120/2120] Creating windows from files...
Created 2120 windows: partida_YD: 535, partida_direta: 519, partida_soft_starter: 524, regime_permanente: 542
Creating features...
[1/2120] Creating features...
[2120/2120] Creating features...
Created features
Job completed (success)
Feature explorer
partida_YD
partida_direta
partida_soft_starter
regime_permanente
On-device performance
PROCESSING TIME
0 ms.
PEAK RAM USAGE
4 KB
© 2025 EdgeImpulse Inc. All rights reserved



Testar diferentes ajustes.

Plataforma Edge Impulse

Testando Dentro da Plataforma


EDGE IMPULSE
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators.
View plans
Test data
Classify all
Set the 'expected outcome' for each sample to the desired outcome to automatically score the impulse.
SAMPLE NAME
EXPECTED OUTCOME
LENGTH
ACCURACY
RESULT
regime_permanent... regime_permanente 1s 100% 1 regime_permanente
partida_YD_995 partida_YD 1s 100% 1 partida_YD
partida_YD_987 partida_YD 1s 100% 1 partida_YD
partida_YD_982 partida_YD 1s 100% 1 partida_YD
partida_YD_973 partida_YD 1s 100% 1 partida_YD
partida_YD_97 partida_YD 1s 100% 1 partida_YD
partida_YD_963 partida_YD 1s 100% 1 partida_YD
partida_YD_958 partida_YD 1s 100% 1 partida_YD
Model testing output
(0)
Classifying data for float32 model...
Job scheduled
Job started
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
Classifying data for Classifier OK
Generating model testing summary...
Finished generating model testing summary
Job completed (success)
Results
Model version:
Unoptimized (float32)
ACCURACY
100.00%
Metrics for Classifier
METRIC
VALUE
Area under ROC Curve 1.00
Weighted average Precision 1.00
Weighted average Recall 1.00
Weighted average F1 score 1.00
Confusion matrix
PARTIDA_YD
PARTIDA_DIRETA
PARTIDA_SOFT_START
REGIME_PERMANENTE
UNCERTAIN
PARTIDA_YD 100% 0% 0% 0% 0%
PARTIDA_DIRETA 0% 100% 0% 0% 0%
PARTIDA_SOFT_STARTER 0% 0% 100% 0% 0%
REGIME_PERMANENTE 0% 0% 0% 100% 0%
F1 SCORE 1.00 1.00 1.00 1.00

ACCURACY
100.00%

Metrics for Classifier

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	1.00
Weighted average Recall	1.00
Weighted average F1 score	1.00

Confusion matrix

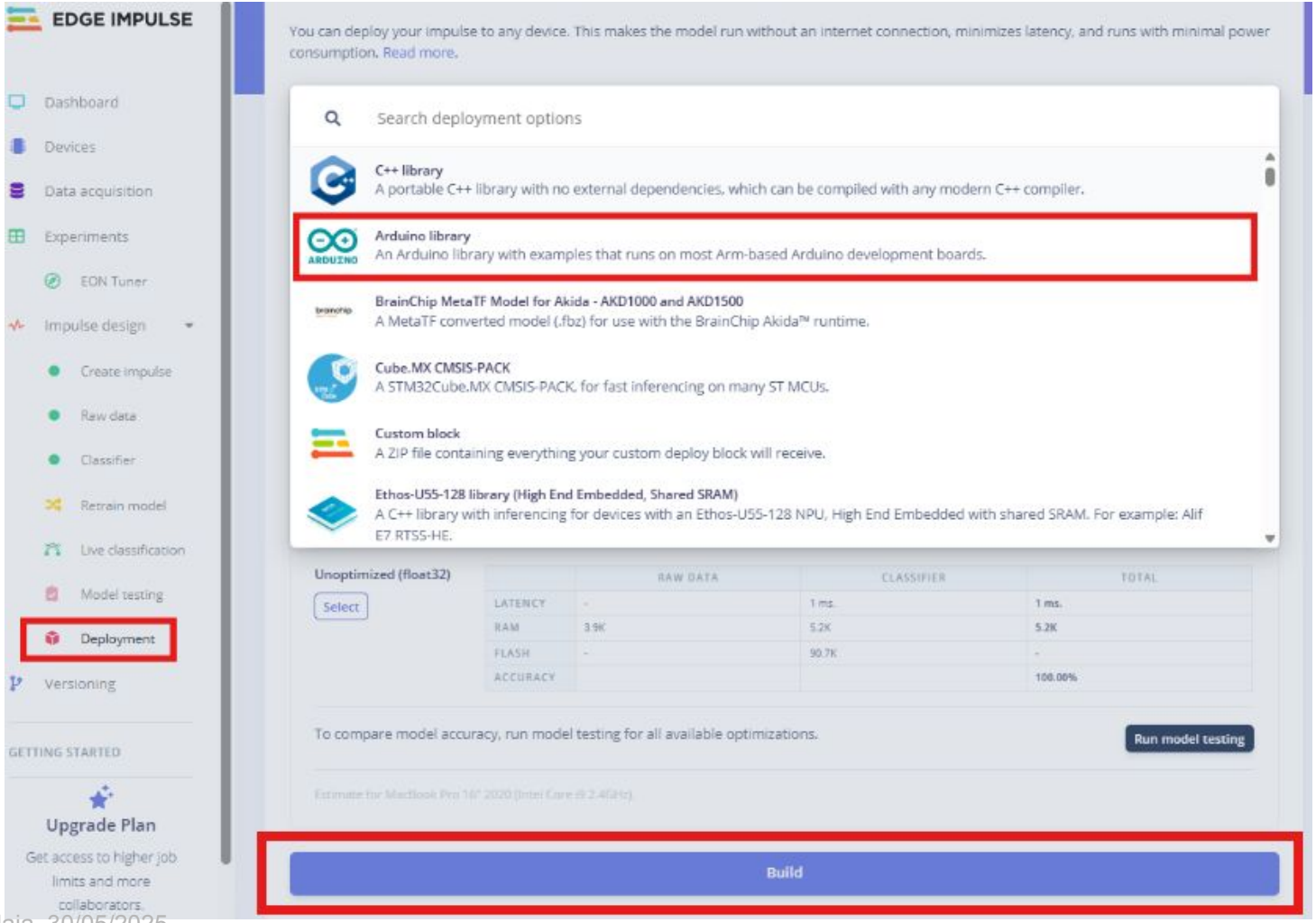
	PARTIDA_YD	PARTIDA_DIRETA	PARTIDA_SOFT_START	REGIME_PERMANENTE	UNCERTAIN
PARTIDA_YD	100%	0%	0%	0%	0%
PARTIDA_DIRETA	0%	100%	0%	0%	0%
PARTIDA_SOFT_STARTER	0%	0%	100%	0%	0%
REGIME_PERMANENTE	0%	0%	0%	100%	0%
F1 SCORE	1.00	1.00	1.00	1.00	

Feature explorer



Plataforma Edge Impulse + ESP32

Embarcando no ESP32


EDGE IMPULSE
Dashboard
Devices
Data acquisition
Experiments
EON Tuner
Impulse design
Create impulse
Raw data
Classifier
Retrain model
Live classification
Model testing
Deployment
Versioning
GETTING STARTED
Upgrade Plan
Get access to higher job limits and more collaborators.
You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. Read more.
Search deployment options
C++ library
A portable C++ library with no external dependencies, which can be compiled with any modern C++ compiler.
Arduino library
An Arduino library with examples that runs on most Arm-based Arduino development boards.
BrainChip MetaTF Model for Akida - AKD1000 and AKD1500
A MetaTF converted model (.fbz) for use with the BrainChip Akida™ runtime.
Cube.MX CMSIS-PACK
A STM32Cube.MX CMSIS-PACK for fast inferencing on many ST MCUs.
Custom block
A ZIP file containing everything your custom deploy block will receive.
Ethos-U55-128 library (High End Embedded, Shared SRAM)
A C++ library with inferencing for devices with an Ethos-U55-128 NPU, High End Embedded with shared SRAM. For example: Alif E7 RTSS-HE.
Unoptimized (float32)
Select
RAW DATA
CLASSIFIER
TOTAL
LATENCY
1 ms.
1 ms.
RAM
3.9K
5.2K
5.2K
FLASH
90.7K
ACCURACY
100.00%
To compare model accuracy, run model testing for all available optimizations.
Run model testing
Build



Built Arduino library

Add this library through the Arduino IDE via:

```
Sketch > Include Library > Add .ZIP Library...
```

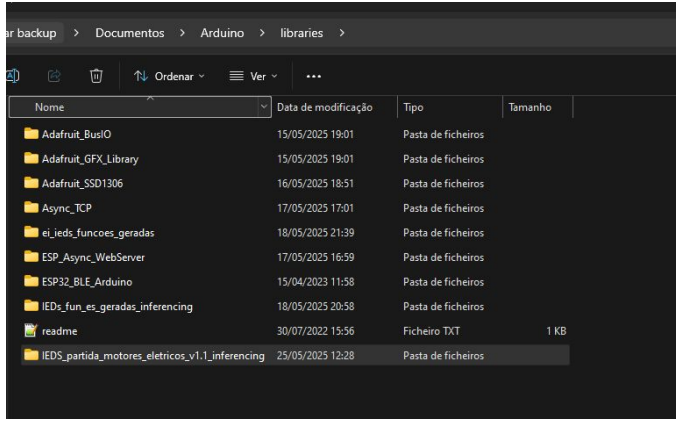
Examples can then be found under:

```
File > Examples > IEDS_partida_motores_eletricos_v1.1_inferencing
```

Download é iniciado automaticamente



Após descompactar, transferir pasta interna do .zip para \Documents\Arduino\libraries



A biblioteca gerada pelo Edge Impulse vem com alguns exemplos de aplicação

```
 ** If the problem persists then there's not enough memory for this model and application.
 */

/* Includes ---------------------------------------------------------------- */
#include <IEDS_partida_motores_eletricos_v1.1_inferencing.h>

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "driver/i2s.h"

/** Audio buffers, pointers and selectors */
typedef struct {
    int16_t *buffer;
    uint8_t buf_ready;
    uint32_t buf_count;
    uint32_t n_samples;
} inference_t;

static inference_t inference;
static const uint32_t sample_buffer_size = 2048;
static signed short sampleBuffer[sample_buffer_size];
static bool debug_nn = false; // Set this to true to see e.g. features generated from the raw signal
static bool record_status = true;

/**
 * @brief      Arduino setup function
 */
void setup()
{
    // put your setup code here, to run once:
    Serial.begin(115200);
```

```ino
void setup()
{
    // put your setup code here, to run once:
    Serial.begin(115200);
    // comment out the below line to cancel the wait for USB connection (needed for native USB)
    while (!Serial);
    Serial.println("Edge Impulse Inferencing Demo");

    // summary of inferencing settings (from model_metadata.h)
    ei_printf("Inferencing settings:\n");
    ei_printf("\tInterval: ");
    ei_printf_float((float)EI_CLASSIFIER_INTERVAL_MS);
    ei_printf(" ms.\n");
    ei_printf("\tFrame size: %d\n", EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE);
    ei_printf("\tSample length: %d ms.\n", EI_CLASSIFIER_RAW_SAMPLE_COUNT / 16);
    ei_printf("\tNo. of classes: %d\n", sizeof(ei_classifier_inferencing_categories) / sizeof(ei_classifier_inferencing_categories[0]));

    ei_printf("\nStarting continious inference in 2 seconds...\n");
    ei_sleep(2000);

    if (microphone_inference_start(EI_CLASSIFIER_RAW_SAMPLE_COUNT) == false) {
        ei_printf("ERR: Could not allocate audio buffer (size %d), this could be due to the window length of your model\r\n", EI_CLASSIFIER_RAW_SAMPLE_COUNT);
        return;
    }

    ei_printf("Recording...\n");
}
```

Plataforma Edge Impulse + ESP32

Exemplo de Aplicação

Exemplo de aplicação

- Classificação de formas de onda
  - Senoidal
  - Quadrada
  - Dente de Serra

Exemplo de aplicação

- Componentes Utilizados
  - Arduino Mega
  - ESP32
  - Display OLED I2C
  - Botões
  - Resistores
  - Protoboard
  - Cabos Jumper
  - Fontes de alimentação

Exemplo de aplicação

- Arduino Mega
  - Responsável por gerar os sinais e realizar a comunicação com o ESP32 através de protocolo de comunicação (UART)

```
void gerarEEnviarAmostra() {
  static int i = 0;
  float t = (float)i / num_amostras;
  float val = 0.0;

  // Geração da forma de onda
  switch (indiceForma) {
    // Senoidal
    case 0: val = sin(2 * PI * t); break;
    // Quadrada
    case 1: val = t < 0.5 ? 1.0 : -1.0; break;
    // Dente de serra
    case 2: val = 2.0 * t - 1.0; break;
  }

  // Ajuste das 3 formas de onda (pico 1, 2 e 3)
  float val_normalizado = (val * pico + pico) / (2.0 * pico);
  int int_val = (int)(val_normalizado * 1023.0);

  Serial1.println(int_val);  // Envia via Serial1 (TX1)
  i = (i + 1) % num_amostras;
}

void setup() {
  //Debug Monitor Serial
  Serial.begin(9600);
  //TX1 e RX1 - comunicação com ESP32
  Serial1.begin(9600);

  pinMode(BTN_FORMA, INPUT_PULLUP);
  pinMode(BTN_PICO, INPUT_PULLUP);
  pinMode(BTN_ENVIA, INPUT_PULLUP);
```

Exemplo de aplicação

- ESP32
  - Recebe comunicação do Arduino
  - Processa o sinal de entrada no modelo de aprendizado de máquina embarcado

```
#include <IEDs_fun_es_geradas_inferencing.h>

#define LED_SENOIDE  2
#define LED_QUADRADA 4
#define LED_SERRA    5
#define RXD2 16
#define TXD2 17

float features[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE];

String ultima_classe_detectada = "";
int contador_repeticoes = 0;
const int LIMIAR_REPETICOES = 3;

bool receberAmostrasSerial(); // Protótipo

void setup() {
  //Debug - Monitor Serial
  Serial.begin(9600);
  //TX2 e RX2 - comunicação com Arduino em UART
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2); // UART com Arduino

  pinMode(LED_SENOIDE, OUTPUT);
  pinMode(LED_QUADRADA, OUTPUT);
  pinMode(LED_SERRA, OUTPUT);

  Serial.println("ESP32 pronto para receber dados e classificar.");
}

void loop() {
  if (!receberAmostrasSerial()) {
    Serial.println("Falha ao receber amostras do Arduino.");
    return;
  }

  signal_t signal;
  if (numpy::signal_from_buffer(features, EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, &signal) != 0) {
    Serial.println("Erro ao criar o sinal.");
    return;
  }

  ei_impulse_result_t result;
  if (run_classifier(&signal, &result, false) != EI_IMPULSE_OK) {
    Serial.println("Erro na inferência.");
```

Exemplo de aplicação

- ESP32
  - Recebe comunicação do Arduino
  - Processa o sinal de entrada no modelo de aprendizado de máquina embarcado

```
  float maxScore = 0;
  String classe_atual = "";
  for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++) {
    if (result.classification[ix].value > maxScore) {
      maxScore = result.classification[ix].value;
      classe_atual = result.classification[ix].label;
    }
  }
  if (classe_atual == ultima_classe_detectada) {
    contador_repeticoes++;
  } else {
    contador_repeticoes = 1;
    ultima_classe_detectada = classe_atual;
  }
  if (contador_repeticoes >= LIMIAR_REPETICOES) {
    digitalWrite(LED_SENOIDE,  classe_atual == "senoide"        ? HIGH : LOW);
    digitalWrite(LED_QUADRADA, classe_atual == "quadrada"       ? HIGH : LOW);
    digitalWrite(LED_SERRA,    classe_atual == "dente_de_serra" ? HIGH : LOW);

    Serial.print("Classe confirmada: ");
    Serial.println(classe_atual);
  } else {
    Serial.print("Classe detectada (ainda não confirmada): ");
    Serial.println(classe_atual);
  }
}
bool receberAmostrasSerial() {
  size_t index = 0;
  unsigned long tempoInicio = millis();

  while (index < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE) {
    if (Serial2.available()) {
      int leitura = Serial2.parseInt();
      if (leitura >= 0 && leitura <= 1023) {
        features[index++] = ((float)leitura / 1023.0f) * 3.3f;
      }
    }
    if (millis() - tempoInicio > 2000) {
      return false;
    }
  }
  return true;
}
```

Exemplo de aplicação

- Resistores
  - Em série com botões para estabilizar sinal
  - Divisor de tensão
    - TX Arduino: 5 Vcc
    - RX ESP32: 3 Vcc

Table 5-1. Absolute Maximum Ratings

Parameter	Description	Min	Max	Unit
VDDA, VDD3P3, VDD3P3_RTC, VDD3P3_CPU, VDD_SDIO	Allowed input voltage	–0.3	3.6	V
I_{output}[1]	Cumulative IO output current	—	1200	mA
T_{STORE}	Storage temperature	–40	150	°C

Fonte: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

Exemplo de aplicação

- Resistores
  - Em série com botões para estabilizar sinal
  - Divisor de tensão (TX do arduino em até 5 Vdc, entrada ESP32 estável entre 0 e 2,2 Vdc)

31.1 DC Characteristics

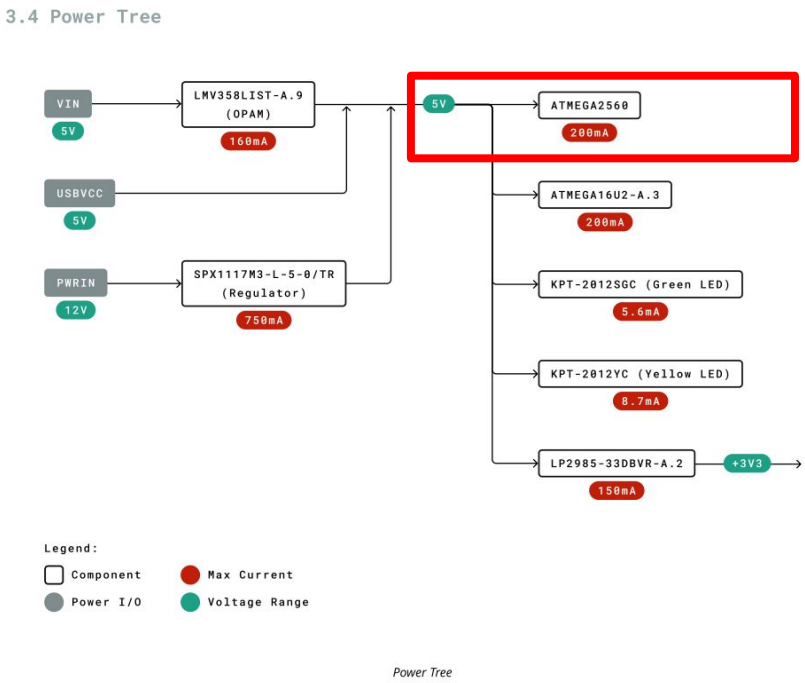
T_A = -40°C to 85°C, V_{CC} = 1.8V to 5.5V (unless otherwise noted)

Symbol	Parameter	Condition	Min.	Typ.	Max.	Units
V_{IL}	Input Low Voltage, Except XTAL1 and Reset pin	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	-0.5 -0.5		$0.2V_{CC}$(1) $0.3V_{CC}$(1)	V
V_{IL1}	Input Low Voltage, XTAL1 pin	V_{CC} = 1.8V - 5.5V	-0.5		$0.1V_{CC}$(1)	
V_{IL2}	Input Low Voltage, RESET pin	V_{CC} = 1.8V - 5.5V	-0.5		$0.1V_{CC}$(1)	
V_{IH}	Input High Voltage, Except XTAL1 and RESET pins	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	$0.7V_{CC}$(2) $0.6V_{CC}$(2)		V_{CC} + 0.5 V_{CC} + 0.5	
V_{IH1}	Input High Voltage, XTAL1 pin	V_{CC} = 1.8V - 2.4V V_{CC} = 2.4V - 5.5V	$0.8V_{CC}$(2) $0.7V_{CC}$(2)		V_{CC} + 0.5 V_{CC} + 0.5	
V_{IH2}	Input High Voltage, RESET pin	V_{CC} = 1.8V - 5.5V	$0.9V_{CC}$(2)		V_{CC} + 0.5	
V_{OL}	Output Low Voltage(3), Except RESET pin	I_{OL} = 20mA, V_{CC} = 5V I_{OL} = 10mA, V_{CC} = 3V			0.9 0.6	
V_{OH}	Output High Voltage(4), Except RESET pin	I_{OH} = -20mA, V_{CC} = 5V I_{OH} = -10mA, V_{CC} = 3V	4.2 2.3			
I_{IL}	Input Leakage Current I/O Pin	V_{CC} = 5.5V, pin low (absolute value)			1	µA
I_{IH}	Input Leakage Current I/O Pin	V_{CC} = 5.5V, pin high (absolute value)			1	
R_{RST}	Reset Pull-up Resistor		30		60	kΩ
R_{PU}	I/O Pin Pull-up Resistor		20		50	

Fonte: microchip.com/downloads/en/DeviceDoc/Atmel-2549-8-bit-AVR-Microcontroller-ATmega640-1280-1281-2560-2561_datasheet.pdf

Exemplo de aplicação

- Resistores
  - Em série com botões para estabilizar sinal
  - Divisor de tensão (TX do arduino em até 5 Vdc, entrada ESP32 estável entre 0 e 2,2 Vdc)

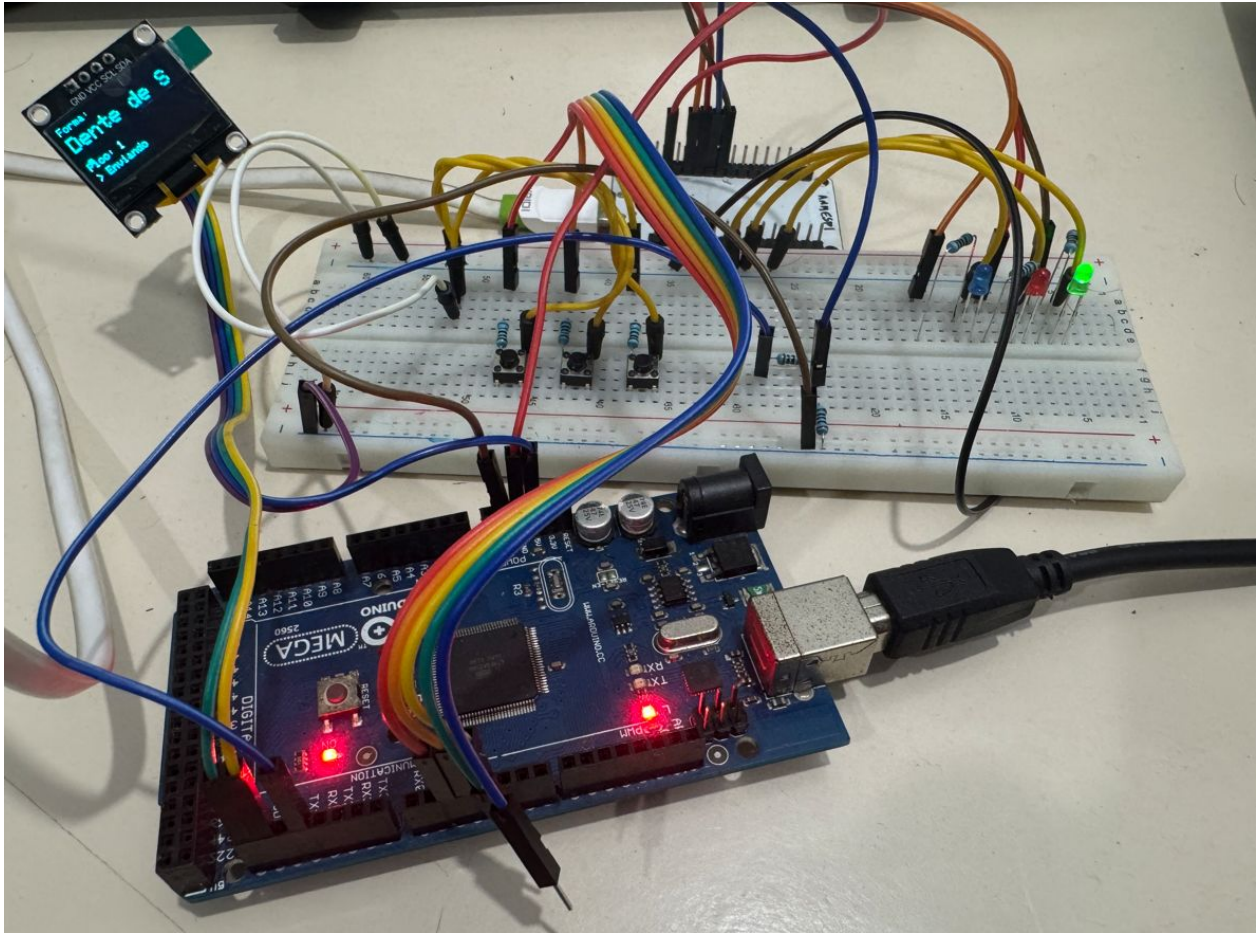


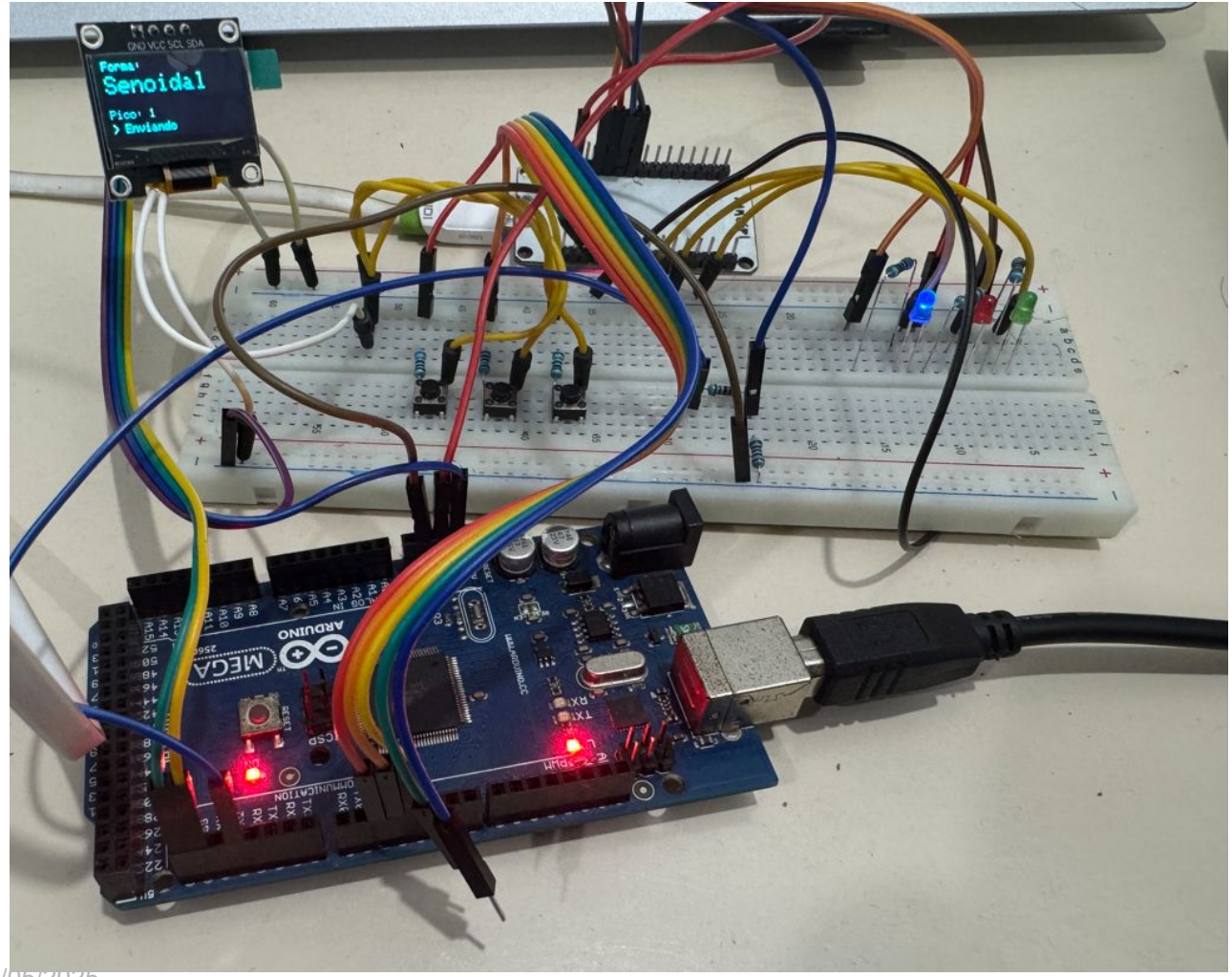
Power Tree

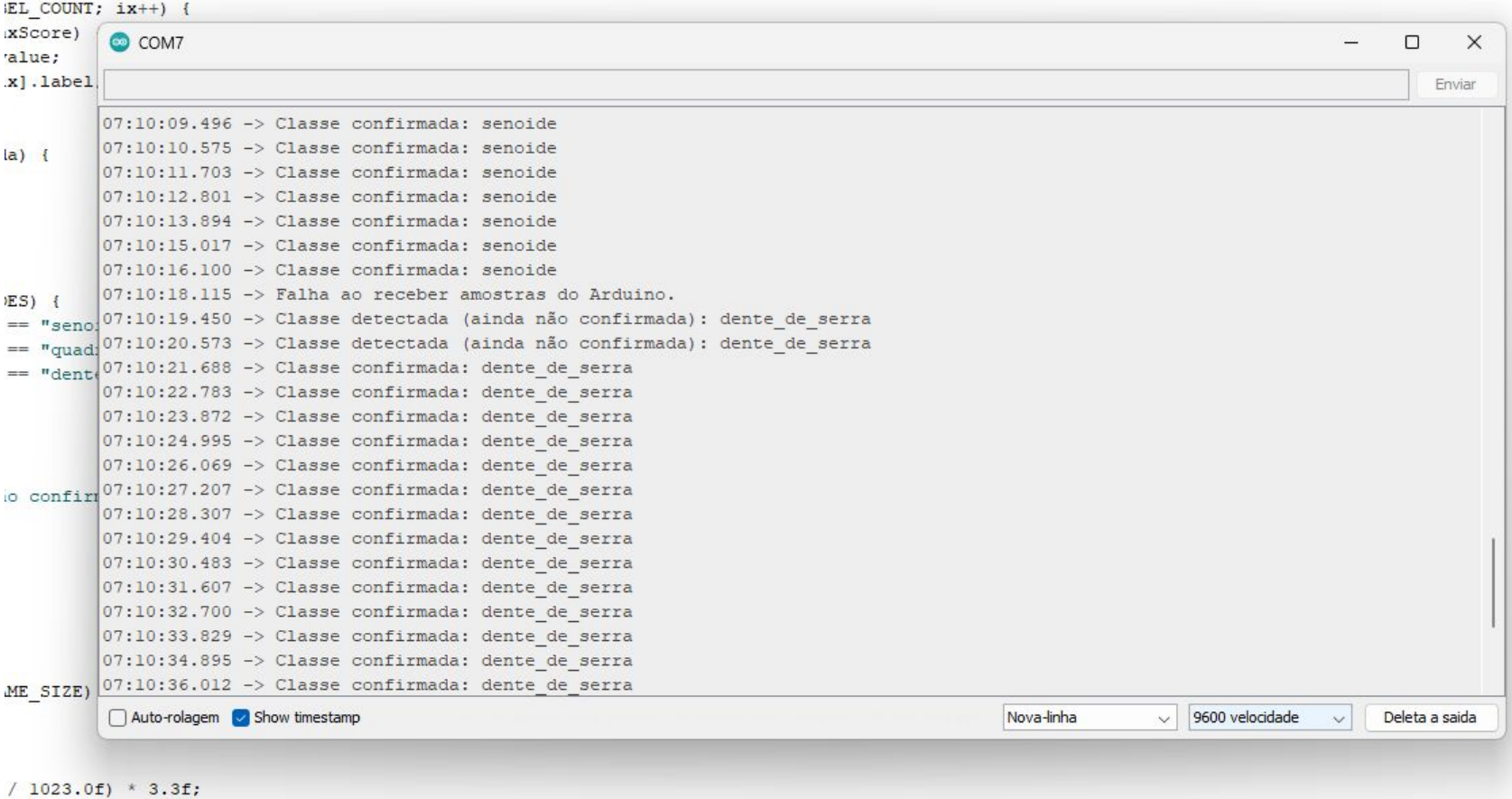
Fonte: docs.arduino.cc/resources/datasheets/A000067-datasheet.pdf

Exemplo de aplicação

- Display e Botões
  - Seleção da forma de onda que será enviada ao ESP32


GND VCC SCL SDA
Forma:
Dente de S
Ang: 1
> Enviando
MEGA
2560
DIGITAL (PWM~)
RESET
POWER
ANALOG IN
COMMUNICATION


GND VCC SCL SDA
Forma:
Senoidal
Pico: 1
> Enviando
ARDUINO
MEGA
2560


EL_COUNT; ix++) {
xScore)
alue;
x].label
la) {
ES) {
== "seno
== "quad
== "dent
io confir
ME_SIZE)
/ 1023.0f) * 3.3f;
COM7
Enviar
07:10:09.496 -> Classe confirmada: senoide
07:10:10.575 -> Classe confirmada: senoide
07:10:11.703 -> Classe confirmada: senoide
07:10:12.801 -> Classe confirmada: senoide
07:10:13.894 -> Classe confirmada: senoide
07:10:15.017 -> Classe confirmada: senoide
07:10:16.100 -> Classe confirmada: senoide
07:10:18.115 -> Falha ao receber amostras do Arduino.
07:10:19.450 -> Classe detectada (ainda não confirmada): dente_de_serra
07:10:20.573 -> Classe detectada (ainda não confirmada): dente_de_serra
07:10:21.688 -> Classe confirmada: dente_de_serra
07:10:22.783 -> Classe confirmada: dente_de_serra
07:10:23.872 -> Classe confirmada: dente_de_serra
07:10:24.995 -> Classe confirmada: dente_de_serra
07:10:26.069 -> Classe confirmada: dente_de_serra
07:10:27.207 -> Classe confirmada: dente_de_serra
07:10:28.307 -> Classe confirmada: dente_de_serra
07:10:29.404 -> Classe confirmada: dente_de_serra
07:10:30.483 -> Classe confirmada: dente_de_serra
07:10:31.607 -> Classe confirmada: dente_de_serra
07:10:32.700 -> Classe confirmada: dente_de_serra
07:10:33.829 -> Classe confirmada: dente_de_serra
07:10:34.895 -> Classe confirmada: dente_de_serra
07:10:36.012 -> Classe confirmada: dente_de_serra
Auto-rolagem
Show timestamp
Nova-linha
9600 velocidade
Deleta a saida

Exemplo de aplicação

- Pontos de atenção
  - Com o filtro errado, tudo pode virar senoide (válido para leitura de sinais analógicos com ADC);
  - Frequência e taxa de amostragem devem ser equivalentes entre os dados utilizados para treinamento, teste e implementação;
  - Tempo de compilação da biblioteca na IED Arduino é bem longo, chegando a cerca de 10 minutos nesse exemplo;